Problem Statement

A university has a limited number of scholarships (here, 3). Many students apply. We must check each applicant against fixed criteria, rank only the eligible ones, and break ties fairly using predefined rules. Doing this by hand is slow and error-prone, so we automate it.

. Objectives

Calculate each student's aggregate (average) percentage.

Check four eligibility rules: aggregate, subject-wise minimum, attendance, and income.

Rank eligible students from best to worst.

Resolve ties with a clear, ordered set of rules.

Award scholarships to the top N and put the rest on a waitlist.

Give a reason for every rejection, so the process is transparent.

Assumptions

Criterion	Rule
Aggregate	≥ 60%
Each subject	≥ 40 marks
Attendance	≥ 75%
Annual family income	≤ ₹3,00,000

Algorithm

1. Define the rule constants (minimum aggregate, subject, attendance, maximum income, seats).
2. Read the list of applicants.
3. For each applicant:
    - Compute the aggregate = total marks ÷ number of subjects.
    - If aggregate < 60 → reject with the reason.
    - If any subject < 40 → reject with the reason.
    - If attendance < 75 → reject with the reason.
    - If income > 3,00,000 → reject with the reason.
    - Otherwise add to the eligible list.
4. Sort the eligible list using the key (−aggregate, −Maths, income, −attendance, name).
5. Assign ranks 1, 2, 3, …

6. Ranks 1 to N → AWARDED; remaining → WAITLIST.
7. Print the rank list and the rejected list with reasons.

Pseudocode

```
CONSTANTS: MIN_AGG=60, MIN_SUBJ=40, MIN_ATT=75, MAX_INCOME=300000,
SEATS=3


FUNCTION aggregate(marks):
  RETURN sum(marks) / count(marks)


FUNCTION check_eligibility(student):
  IF aggregate(student.marks) < MIN_AGG      RETURN (False, "low aggregate")
  FOR each mark in student.marks:
    IF mark < MIN_SUBJ                  RETURN (False, "low subject mark")
  IF student.attendance < MIN_ATT           RETURN (False, "low attendance")
  IF student.income > MAX_INCOME            RETURN (False, "high income")
  RETURN (True, "")


FUNCTION ranking_key(student):
  RETURN (-aggregate, -maths_marks, income, -attendance, name)


MAIN:
  eligible = [], rejected = []
  FOR each student in applicants:
    (ok, reason) = check_eligibility(student)
    IF ok: add student to eligible
    ELSE:  add (student, reason) to rejected
  SORT eligible BY ranking_key
  FOR rank = 1 to length(eligible):
    IF rank <= SEATS: status = "AWARDED"
    ELSE:           status = "WAITLIST"
    PRINT rank, student, status
```

PRINT rejected list

Python source code

```python
# University Scholarship Allocation System

MIN_AGGREGATE = 60.0
MIN_SUBJECT   = 40
MIN_ATTENDANCE = 75.0
MAX_INCOME    = 300000
TOTAL_SEATS   = 3
TIE_SUBJECT   = "Maths"


def calculate_aggregate(marks):
    """Average of all subject marks (each out of 100)."""
    return sum(marks.values()) / len(marks)


def check_eligibility(student):
    """Return (True, '') if eligible, else (False, reason)."""
    agg = calculate_aggregate(student["marks"])
    if agg < MIN_AGGREGATE:
        return False, f"Aggregate {agg:.2f}% < {MIN_AGGREGATE}%"
    for subject, mark in student["marks"].items():
        if mark < MIN_SUBJECT:
            return False, f"{subject} mark {mark} < {MIN_SUBJECT}"
    if student["attendance"] < MIN_ATTENDANCE:
        return False, f"Attendance {student['attendance']}% < {MIN_ATTENDANCE}%"
    if student["income"] > MAX_INCOME:
        return False, f"Income {student['income']} > {MAX_INCOME}"
    return True, ""


def ranking_key(student):
    """
    Sort key. Python sorts smallest first, so we negate
    values where a HIGHER number is better.
    Tie-break order:
      1. Higher aggregate
      2. Higher marks in TIE_SUBJECT
      3. Lower family income
      4. Higher attendance
      5. Name (alphabetical) - final fallback
    """
    return (
        -calculate_aggregate(student["marks"]),
        -student["marks"].get(TIE_SUBJECT, 0),
        student["income"],
        -student["attendance"],
        student["name"],
    )
```

```python
def allocate_scholarships(students):
    eligible, rejected = [], []
    for s in students:
        ok, reason = check_eligibility(s)
        if ok:
            eligible.append(s)
        else:
            rejected.append((s["name"], reason))
    eligible.sort(key=ranking_key)
    return eligible, rejected


def print_report(students):
    eligible, rejected = allocate_scholarships(students)
    print("-" * 72)
    print("SCHOLARSHIP RANK LIST")
    print("=" * 72)
    print(f"{'Rank':<5}{'Name':<10}{'Agg%':<8}{TIE_SUBJECT:<7}{'Income':<10}{'Att%':<7}Status")
    print("-" * 72)
    for rank, s in enumerate(eligible, start=1):
        status = "AWARDED" if rank <= TOTAL_SEATS else "WAITLIST"
        print(f"{rank:<5}{s['name']:<10}{calculate_aggregate(s['marks']):<8.2f}"
              f"{s['marks'].get(TIE_SUBJECT, 0):<7}{s['income']:<10}{s['attendance']:<7}{status}")
    print("\nREJECTED APPLICANTS")
    print("-" * 72)
    for name, reason in rejected:
        print(f"{name:<10}-> {reason}")
    return eligible, rejected


if __name__ == "__main__":
    students = [
        {"name": "Asha",  "marks": {"Maths": 90, "Physics": 85, "English": 80}, "attendance": 92, "income": 150000},
        {"name": "Bharat","marks": {"Maths": 85, "Physics": 90, "English": 80}, "attendance": 88, "income": 120000},
        {"name": "Charan","marks": {"Maths": 95, "Physics": 80, "English": 80}, "attendance": 90, "income": 200000},
        {"name": "Divya", "marks": {"Maths": 90, "Physics": 85, "English": 80}, "attendance": 92, "income": 100000},
        {"name": "Esha",  "marks": {"Maths": 95, "Physics": 95, "English": 35}, "attendance": 95, "income": 100000},
        {"name": "Farid", "marks": {"Maths": 70, "Physics": 65, "English": 60}, "attendance": 70, "income": 100000},
        {"name": "Gita",  "marks": {"Maths": 95, "Physics": 95, "English": 95}, "attendance": 95, "income": 500000},
        {"name": "Hari",  "marks": {"Maths": 50, "Physics": 55, "English": 60}, "attendance": 90, "income": 100000},
        {"name": "Indu",  "marks": {"Maths": 80, "Physics": 80, "English": 80}, "attendance": 85, "income": 250000},
    ]
    print_report(students)
```

Test Cases

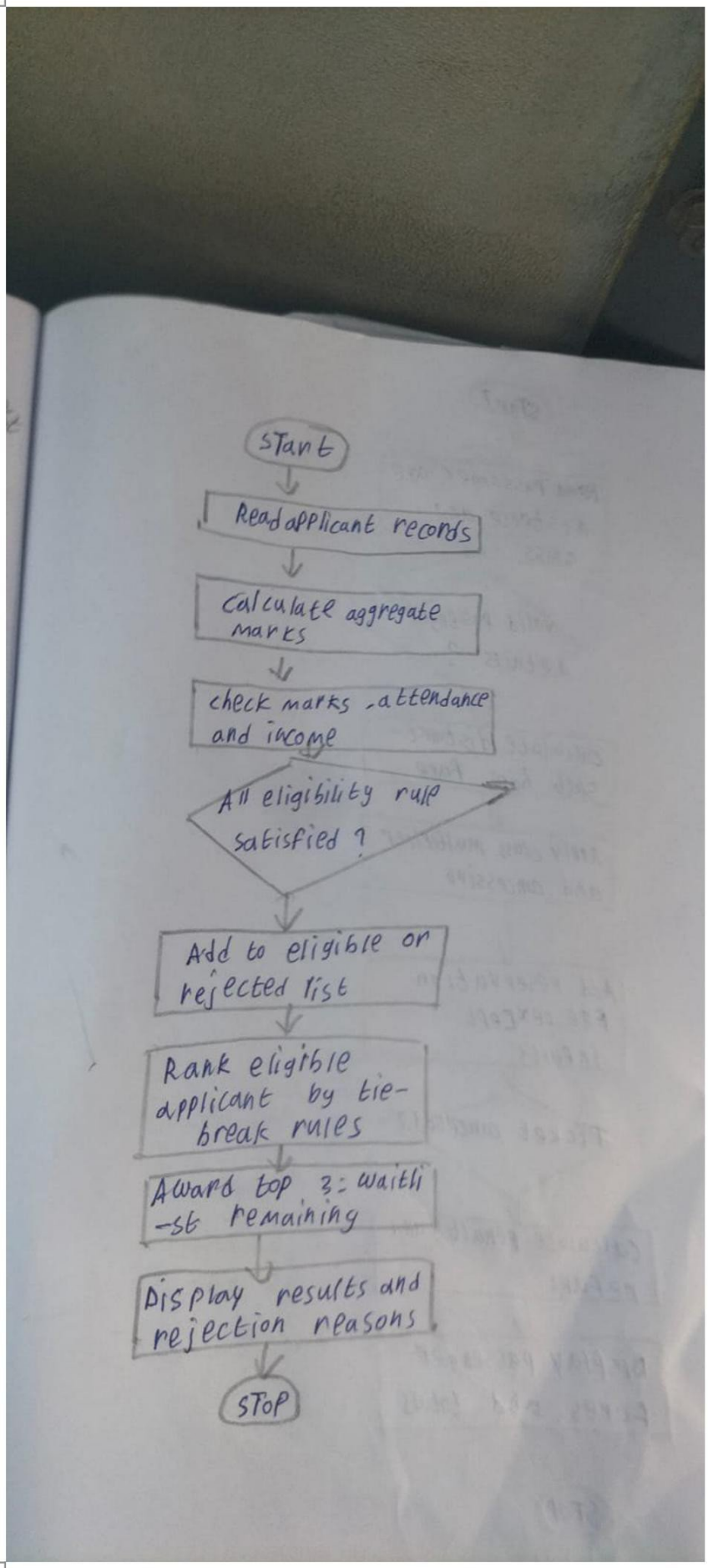
Student	Input highlights	Expected result	Rule tested
Esha	Marks 95, 95, 35; aggregate 75	Rejected: English 35 < 40	Subject-wise minimum
Farid	Attendance 70%	Rejected: attendance	Attendance rule
Gita	Income ₹5,00,000, top marks	Rejected: income	Income rule
Hari	Aggregate 55%	Rejected: aggregate	Aggregate rule
Asha, Bharat, Charan, Divya	All aggregate 85	Ordered by tie-breakers	Tie resolution
Charan vs Divya	Same aggregate; Maths 95 vs 90	Charan ranks above Divya	Tie-break rule 2 (Maths)
Divya vs Asha	Same aggregate and Maths; income ₹1L vs ₹1.5L	Divya above Asha	Tie-break rule 3 (income)
Asha, Bharat, Charan, Divya, Indu	5 eligible, 3 seats	Top 3 awarded, rest waitlisted	Seat limit

Output

```
======================================================================
SCHOLARSHIP RANK LIST
======================================================================
Rank Name      Agg%    Maths  Income    Att%   Status
----------------------------------------------------------------------
1    Charan    85.00   95     200000    90     AWARDED
2    Divya     85.00   90     100000    92     AWARDED
3    Asha      85.00   90     150000    92     AWARDED
4    Bharat    85.00   85     120000    88     WAITLIST
5    Indu      80.00   80     250000    85     WAITLIST

REJECTED APPLICANTS
----------------------------------------------------------------------
Esha      -> English mark 35 < 40
Farid     -> Attendance 70% < 75.0%
Gita      -> Income 500000 > 300000
Hari      -> Aggregate 55.00% < 60.0%
```

Flowchart


Start
Read applicant records
Calculate aggregate marks
check marks, attendance and income
All eligibility rule satisfied ?
Add to eligible or rejected list
Rank eligible applicant by tie-break rules
Award top 3: waitli -st remaining
Display results and rejection reasons
STOP

Conclusion

The system filters applicants with four eligibility rules, ranks the eligible ones, and uses a fixed, ordered set of tie-break rules to decide between equal candidates. Every rejection comes with a reason, which keeps the process transparent and fair. All rules are stored as constants at the top of the program, so the university can change the policy without touching the logic.

Problem Statement

A railway booking counter must calculate the ticket price for a **group of passengers** travelling together. The price depends on:

the **distance** travelled (longer distance costs less per km),

the **travel class** (Sleeper, AC 3 Tier, AC 2 Tier, AC First),

the passenger's **age and gender** (concessions for children and senior citizens),

a **reservation fee** per berth.

Objectives

Calculate the base fare using **distance slabs**.

Apply a **class multiplier** for the chosen travel class.

Apply **age/gender-based concessions** to each passenger.

Add the **reservation fee** (free infants get no berth, so no fee).

Show a passenger-wise fare table with a grand total.

Calculate the **cancellation penalty and refund** for each passenger.

Assumptions

Distance	**Rate per km**
First 100 km	₹1.00
Next 200 km (101–300)	₹0.80
Above 300 km	₹0.60

Class	**Fare multiplier**	**Reservation fee**	**Cancellation clerkage fee**

SL (Sleeper)	1.0	₹20	₹120
3A (AC 3 Tier)	2.2	₹40	₹180
2A (AC 2 Tier)	3.0	₹50	₹200
1A (AC First)	4.5	₹60	₹240

Passenger	Concession
Age below 5	100% (free, no berth)
Age 5–11 (child)	50%
Woman aged 58+	50%
Man aged 60+	40%
Others	0%

Time before departure	Penalty
48 hours or more	Flat clerkage fee (never more than the fare)
12 to 48 hours	25% of fare
4 to 12 hours	50% of fare
Less than 4 hours	100% of fare (no refund)

Algorithm

1. Read the passenger list, travel class and distance.
2. **Base fare:** go through each slab and add (km in that slab × slab rate). If the result is below ₹30, use ₹30.
3. For each passenger:
    1. Class fare = base fare × class multiplier.
    2. Find the concession % from age and gender.
    3. Fare after concession = class fare − (class fare × concession %).
    4. If concession is 100%, reservation fee = 0, otherwise the class reservation fee.
    5. Passenger total = fare + reservation fee.
4. Grand total = sum of all passenger totals.
5. **On cancellation:** for each passenger, pick the penalty by the hours rule, then Refund = fare − penalty (never below 0). The reservation fee is not refunded.
6. Print the fare table, grand total and refund table.

Pseudocode

```
CONSTANTS: SLABS, CLASSES, MIN_FARE = 30


FUNCTION base_fare(distance):
  fare = 0, previous_limit = 0
  FOR each (limit, rate) in SLABS:
    IF distance > previous_limit:
      km_in_slab = MIN(distance, limit) - previous_limit
      fare = fare + km_in_slab * rate
    previous_limit = limit
  RETURN MAX(fare, MIN_FARE)


FUNCTION concession(age, gender):
  IF age < 5                RETURN 100
```

```
    IF age <= 11                  RETURN 50
    IF gender == F AND age >= 58     RETURN 50
    IF gender == M AND age >= 60     RETURN 40
    RETURN 0

FUNCTION passenger_fare(p, class, distance):
    class_fare = base_fare(distance) * multiplier[class]
    percent = concession(p.age, p.gender)
    fare = class_fare - class_fare * percent / 100
    IF percent == 100: reservation = 0 ELSE reservation = res_fee[class]
    RETURN fare, reservation, fare + reservation

FUNCTION penalty(fare, class, hours):
    IF hours >= 48:  RETURN MIN(clerkage[class], fare)
    IF hours >= 12:  RETURN 0.25 * fare
    IF hours >= 4:   RETURN 0.50 * fare
    RETURN fare

MAIN:
    FOR each passenger: compute fare details
    grand_total = SUM of passenger totals
    PRINT table and grand_total
    IF cancellation requested:
        FOR each passenger:
            refund = MAX(fare - penalty(fare, class, hours), 0)
        PRINT refund table
```

Python source code

```python
# Railway Fare and Reservation  Calculator

SLABS = [(100, 1.00), (300, 0.80), (float("inf"), 0.60)]
CLASSES = {
    "SL": (1.0, 20, 120),
    "3A": (2.2, 40, 180),
    "2A": (3.0, 50, 200),
    "1A": (4.5, 60, 240),
}

MIN_FARE = 30


def base_fare(distance):
    """Slab-wise fare for a Sleeper-class full ticket."""
    fare, previous_limit = 0.0, 0
    for limit, rate in SLABS:
        if distance > previous_limit:
            km_in_slab = min(distance, limit) - previous_limit
            fare += km_in_slab * rate
        previous_limit = limit
    return max(fare, MIN_FARE)


def concession_percent(age, gender):
    """Return (concession %, category name)."""
    if age < 5:
        return 100, "Infant (free)"
    if age <= 11:
        return 50, "Child"
    if gender == "F" and age >= 58:
        return 50, "Senior Woman"
    if gender == "M" and age >= 60:
        return 40, "Senior Man"
    return 0, "Adult"
```

```python
def passenger_fare(passenger, travel_class, distance):
    """Fare details for ONE passenger."""
    multiplier, res_fee, _ = CLASSES[travel_class]
    class_fare = base_fare(distance) * multiplier
    percent, category = concession_percent(passenger["age"], passenger["gender"]
    discount = class_fare * percent / 100
    fare_after = class_fare - discount
    reservation = 0 if percent == 100 else res_fee
    return {
        "name": passenger["name"], "age": passenger["age"], "category": category
        "class_fare": round(class_fare, 2), "discount": round(discount, 2),
        "fare": round(fare_after, 2), "reservation": reservation,
        "total": round(fare_after + reservation, 2),
    }

def calculate_booking(passengers, travel_class, distance):
    details = [passenger_fare(p, travel_class, distance) for p in passengers]
    return {"class": travel_class, "distance": distance, "details": details,
            "grand_total": round(sum(d["total"] for d in details), 2)}


def cancellation_penalty(detail, travel_class, hours_before):
    """Penalty for one passenger. Reservation fee is never refunded."""
    if detail["total"] == 0:
        return 0.0
    clerkage = CLASSES[travel_class][2]
    if hours_before >= 48:
        penalty = min(clerkage, detail["fare"])
    elif hours_before >= 12:
        penalty = detail["fare"] * 0.25
    elif hours_before >= 4:
        penalty = detail["fare"] * 0.50
    else:
        penalty = detail["fare"]
    return round(penalty, 2)
```

```python
def cancel_booking(booking, hours_before):
    rows, total_refund = [], 0.0
    for d in booking["details"]:
        penalty = cancellation_penalty(d, booking["class"], hours_before)
        refund = max(d["fare"] - penalty, 0)
        rows.append((d["name"], d["total"], penalty, d["reservation"], round(refund, 2)))
        total_refund += refund
    return rows, round(total_refund, 2)


def print_booking(booking):
    print(f"Class: {booking['class']}   Distance: {booking['distance']} km   "
          f"Base fare (Sleeper): Rs {base_fare(booking['distance']):.2f}")
    print("-" * 78)
    print(f"{'Name':<9}{'Age':<5}{'Category':<15}{'ClassFare':<11}{'Discount':<10}{'Fare':<9}{'ResFee':<8}{'Total'}")
    print("-" * 78)
    for d in booking["details"]:
        print(f"{d['name']:<9}{d['age']:<5}{d['category']:<15}{d['class_fare']:<11.2f}"
              f"{d['discount']:<10.2f}{d['fare']:<9.2f}{d['reservation']:<8}{d['total']:.2f}")
    print("-" * 78)
    print(f"GRAND TOTAL = Rs {booking['grand_total']:.2f}")


def print_cancellation(booking, hours):
    rows, refund = cancel_booking(booking, hours)
    print(f"\nCancellation {hours} hours before departure")
    print("-" * 66)
    print(f"{'Name':<9}{'Paid':<10}{'Penalty':<10}{'ResFee(no refund)':<19}{'Refund'}")
    for name, paid, penalty, res, ref in rows:
        print(f"{name:<9}{paid:<10.2f}{penalty:<10.2f}{res:<19}{ref:.2f}")
    print(f"TOTAL REFUND = Rs {refund:.2f}")
```

```python
if __name__ == "__main__":
    family = [
        {"name": "Ravi",    "age": 35, "gender": "M"},
        {"name": "Sita",    "age": 32, "gender": "F"},
        {"name": "Kiran",   "age": 8,  "gender": "M"},
        {"name": "Baby",    "age": 2,  "gender": "F"},
        {"name": "Grandpa", "age": 65, "gender": "M"},
        {"name": "Granny",  "age": 60, "gender": "F"},
    ]
    print("=" * 78)
    print("RAILWAY FARE CALCULATOR - Booking 1")
    print("=" * 78)
    booking = calculate_booking(family, "3A", 450)
    print_booking(booking)
    for h in (72, 24, 6, 2):
        print_cancellation(booking, h)

    print("\n" + "=" * 78)
    print("Booking 2: single adult, short trip (minimum fare check)")
    print("=" * 78)
    print_booking(calculate_booking([{"name": "Amit", "age": 25, "gender": "M"}], "SL", 20))
```

Test Cases

#	Input	Expected result	What it tests
1	Distance 0 or 1 km	Base fare = ₹30	Minimum fare

2	Distance 50 km	₹50.00	Within the first slab
3	Distance 100 km	₹100.00	Slab boundary
4	Distance 101 km	₹100.80	Crossing into slab 2
5	Distance 300 km	₹260.00	Second slab boundary
6	Distance 450 km	₹350.00	All three slabs together
7	3A, 450 km, adult age 35	Fare 770 + 40 = ₹810	Class multiplier (350 × 2.2)
8	Child age 8	50% off fare, fee charged	Child concession
9	Infant age 2	Total ₹0, no reservation fee	Free travel
10	Man age 65 / Woman age 60	40% / 50% off	Senior concessions
11	Man age 59	No concession (adult)	Age boundary
12	Cancel 72 h before, adult	Penalty ₹180, refund ₹590	Flat clerkage fee
13	Cancel 24 h before, adult	Penalty 25% of 770 = ₹192.50	25% rule

14	Cancel 6 h before, adult	Penalty 50% of 770 = ₹385	50% rule
15	Cancel 2 h before	Refund ₹0	No refund rule
16	Cancel exactly 48 h / 12 h / 4 h before	Falls in the earlier-hours rule (≥ is used)	Boundary hours

Output

```
================================================================================
RAILWAY FARE CALCULATOR - Booking 1
================================================================================
Class: 3A   Distance: 450 km   Base fare (Sleeper): Rs 350.00
--------------------------------------------------------------------------------
Name     Age  Category       ClassFare  Discount  Fare     ResFee  Total
--------------------------------------------------------------------------------
Ravi     35   Adult          770.00     0.00      770.00   40      810.00
Sita     32   Adult          770.00     0.00      770.00   40      810.00
Kiran    8    Child          770.00     385.00    385.00   40      425.00
Baby     2    Infant (free)  770.00     770.00    0.00     0       0.00
Grandpa  65   Senior Man     770.00     308.00    462.00   40      502.00
Granny   60   Senior Woman   770.00     385.00    385.00   40      425.00
--------------------------------------------------------------------------------
GRAND TOTAL = Rs 2972.00

Cancellation 72 hours before departure
------------------------------------------------------------------
Name     Paid      Penalty   ResFee(no refund)  Refund
Ravi     810.00    180.00    40                 590.00
Sita     810.00    180.00    40                 590.00
Kiran    425.00    180.00    40                 205.00
Baby     0.00      0.00      0                  0.00
Grandpa  502.00    180.00    40                 282.00
Granny   425.00    180.00    40                 205.00
TOTAL REFUND = Rs 1872.00
```

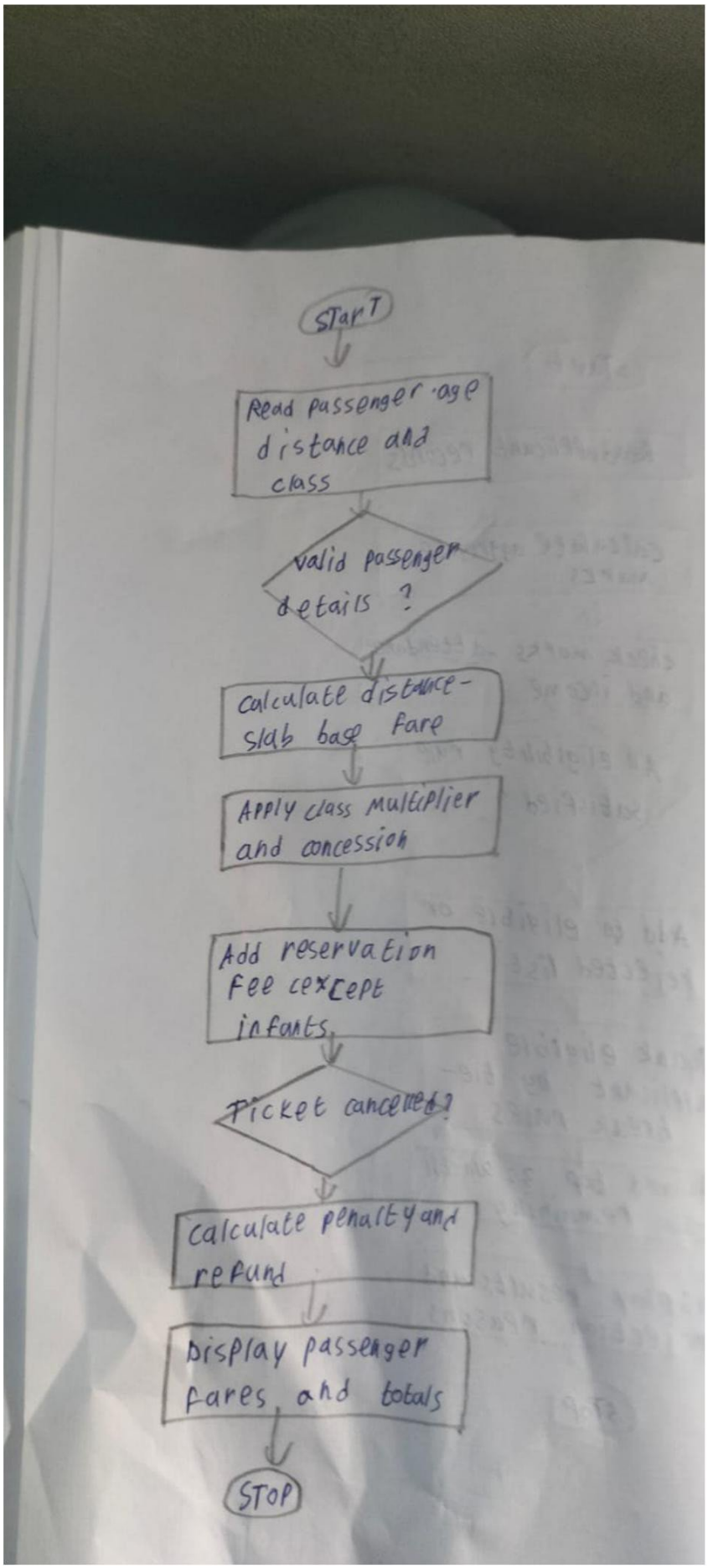
```
Cancellation 24 hours before departure
-----------------------------------------------------------------
Name     Paid      Penalty   ResFee(no refund)  Refund
Ravi     810.00    192.50    40                 577.50
Sita     810.00    192.50    40                 577.50
Kiran    425.00    96.25     40                 288.75
Baby     0.00      0.00      0                  0.00
Grandpa  502.00    115.50    40                 346.50
Granny   425.00    96.25     40                 288.75
TOTAL REFUND = Rs 2079.00

Cancellation 6 hours before departure
-----------------------------------------------------------------
Name     Paid      Penalty   ResFee(no refund)  Refund
Ravi     810.00    385.00    40                 385.00
Sita     810.00    385.00    40                 385.00
Kiran    425.00    192.50    40                 192.50
Baby     0.00      0.00      0                  0.00
Grandpa  502.00    231.00    40                 231.00
Granny   425.00    192.50    40                 192.50
TOTAL REFUND = Rs 1386.00

Cancellation 2 hours before departure
-----------------------------------------------------------------
Name     Paid      Penalty   ResFee(no refund)  Refund
Ravi     810.00    770.00    40                 0.00
Sita     810.00    770.00    40                 0.00
Kiran    425.00    385.00    40                 0.00
Baby     0.00      0.00      0                  0.00
Grandpa  502.00    462.00    40                 0.00
Granny   425.00    385.00    40                 0.00
TOTAL REFUND = Rs 0.00
```

```
================================================================================
Booking 2: single adult, short trip (minimum fare check)
================================================================================
Class: SL   Distance: 20 km   Base fare (Sleeper): Rs 30.00
--------------------------------------------------------------------------------
Name     Age  Category       ClassFare  Discount  Fare     ResFee  Total
--------------------------------------------------------------------------------
Amit     20   Adult          30.00      0.00      30.00    20      50.00
--------------------------------------------------------------------------------
GRAND TOTAL = Rs 50.00
```

Flowchart


START
Read passenger age distance and class
Valid passenger details ?
Calculate distance-slab base fare
Apply class multiplier and concession
Add reservation fee except infants
Ticket cancelled?
Calculate penalty and refund
Display passenger fares and totals
STOP

Conclusion

The calculator combines four ideas: slab-wise distance fares, class multipliers, age-based concessions and a time-based cancellation penalty. Each passenger is calculated separately, so mixed groups (adults, children, seniors, infants) are handled correctly. All rates sit in constants at the top of the program, so the railway can change policy without touching the logic.

Problem Statement

A university offers several courses, and each course has its own rules: which subjects a student must have studied, the minimum marks in those subjects, an overall aggregate cutoff, and a fixed number of seats. Hundreds of candidates apply, and each wants one course. We must **check every candidate against their course's rules**, **rank the eligible ones** , **break ties with fixed criteria** , and produce a **ranked list** showing who is admitted and who is waitlisted.

Objectives

Store each course's rules (required subjects, minimum marks, cutoff, seats).
Check **subject eligibility** : the candidate must have studied each required subject and scored its minimum.
Check the **aggregate** against the course-specific cutoff.
Rank the eligible candidates of each course separately.
Break ties using predefined criteria.
Mark candidates as **ADMITTED** (within seats) or **WAITLIST** .
Give a clear reason for every rejection.

Assumptions

Course	Required subjects (minimum marks)	Cutoff	Seats	Tie-break subject
CSE	Maths 60, Physics 50, English 40	70%	3	Maths
PHY	Maths 50, Physics 60	60%	2	Physics
COM	Accountancy 50, English 40	55%	2	Accountancy

Algorithm

1. Define the course rules.
2. Read the list of candidates.
3. For each candidate:
    a. If the course does not exist → invalid application.
    b. For each required subject: if it is missing or below its minimum → reject with reason.
    c. Compute the aggregate. If it is below the course cutoff → reject with reason.
    d. Otherwise add the candidate to that course's **eligible list** .
4. For each course, sort its eligible list using the tie-break key.

5. Assign ranks 1, 2, 3, …
6. Rank ≤ seats → **ADMITTED**; otherwise → **WAITLIST**.
7. Print the ranked list and rejection reasons for every course.

Pseudocode

```
COURSES = { code: required_subjects, cutoff, seats, primary_subject }

FUNCTION aggregate(marks):
   RETURN sum(marks) / count(marks)

FUNCTION check_eligibility(candidate):
   course = COURSES[candidate.course]
   FOR each (subject, minimum) in course.required:
      IF subject not in candidate.marks   RETURN (False, "subject missing")
      IF candidate.marks[subject] < minimum RETURN (False, "low subject mark")
   IF aggregate(candidate.marks) < course.cutoff RETURN (False, "low aggregate")
   RETURN (True, "")

FUNCTION ranking_key(candidate):
   RETURN (-aggregate, -primary_subject_marks, -entrance, dob, name)

MAIN:
   FOR each candidate:
      IF candidate.course not in COURSES: mark invalid; CONTINUE
      (ok, reason) = check_eligibility(candidate)
      IF ok: add to eligible[course]
      ELSE:  add (candidate, reason) to rejected[course]
   FOR each course:
      SORT eligible[course] BY ranking_key
      FOR rank = 1 to length(eligible[course]):
         IF rank <= seats: status = "ADMITTED"
         ELSE:           status = "WAITLIST"
         PRINT rank, candidate, status
      PRINT rejected list
```

Python source code

```python
# University Admission Decision Engine

COURSES = {
    "CSE": {
        "required": {"Maths": 60, "Physics": 50, "English": 40},
        "cutoff": 70.0,
        "seats": 3,
        "primary": "Maths",
    },
    "PHY": {
        "required": {"Maths": 50, "Physics": 60},
        "cutoff": 60.0,
        "seats": 2,
        "primary": "Physics",
    },
    "COM": {
        "required": {"Accountancy": 50, "English": 40},
        "cutoff": 55.0,
        "seats": 2,
        "primary": "Accountancy",
    },
}


def calculate_aggregate(marks):
    """Average of all subject marks (each out of 100)."""
    return sum(marks.values()) / len(marks)


def check_eligibility(candidate):
    """Return (True, '') if eligible, else (False, reason)."""
    course = COURSES.get(candidate["course"])
    if course is None:
        return False, f"Unknown course '{candidate['course']}'"


    for subject, minimum in course["required"].items():
        if subject not in candidate["marks"]:
            return False, f"Required subject {subject} not studied"
        if candidate["marks"][subject] < minimum:
            return False, f"{subject} {candidate['marks'][subject]} < {minimum}"
```

```python
    agg = calculate_aggregate(candidate["marks"])
    if agg < course["cutoff"]:
        return False, f"Aggregate {agg:.2f}% < cutoff {course['cutoff']}%"
    return True, ""


def ranking_key(candidate):
    """
    Sort key (smallest first, so negate 'higher is better' values).
    Tie-break order:
      1. Higher aggregate
      2. Higher marks in the course's primary subject
      3. Higher entrance test score
      4. Older candidate (earlier date of birth)
      5. Name (alphabetical) - final fallback
    """
    course = COURSES[candidate["course"]]
    return (
        -calculate_aggregate(candidate["marks"]),
        -candidate["marks"][course["primary"]],
        -candidate["entrance"],
        candidate["dob"],
        candidate["name"],
    )
```

```python
def run_admission(candidates):
    """Return {course: {'ranked': [...], 'rejected': [...]}}"""
    result = {c: {"ranked": [], "rejected": []} for c in COURSES}
    unknown = []
    for cand in candidates:
        ok, reason = check_eligibility(cand)
        if cand["course"] not in COURSES:
            unknown.append((cand["name"], reason))
        elif ok:
            result[cand["course"]]["ranked"].append(cand)
        else:
            result[cand["course"]]["rejected"].append((cand["name"], reason))
    for course in result:
        result[course]["ranked"].sort(key=ranking_key)
    return result, unknown


def print_report(candidates):
    result, unknown = run_admission(candidates)
    for code, data in result.items():
        info = COURSES[code]
        print("=" * 76)
        print(f"COURSE: {code}   Seats: {info['seats']}   Cutoff: {info['cutoff']}%   "
              f"Tie-break subject: {info['primary']}")
        print("=" * 76)
        print(f"{'Rank':<5}{'Name':<9}{'Agg%':<8}{info['primary'][:7]:<9}{'Entrance':<10}{'DOB':<12}Status")
        print("-" * 76)
        if not data["ranked"]:
            print("(no eligible candidates)")
        for rank, c in enumerate(data["ranked"], start=1):
            status = "ADMITTED" if rank <= info["seats"] else "WAITLIST"
            print(f"{rank:<5}{c['name']:<9}{calculate_aggregate(c['marks']):<8.2f}"
                  f"{c['marks'][info['primary']]:<9}{c['entrance']:<10}{c['dob']:<12}{status}")
        if data["rejected"]:
            print("\nRejected:")
            for name, reason in data["rejected"]:
                print(f"  {name:<9}-> {reason}")
        print()
    for name, reason in unknown:
        print(f"Invalid application: {name} -> {reason}")
```

```python
if __name__ == "__main__":
    candidates = [

        {"name": "Arun",   "course": "CSE", "marks": {"Maths": 90, "Physics": 80, "English": 70}, "entrance": 88, "dob": "2006-03-10"},
        {"name": "Bhavna", "course": "CSE", "marks": {"Maths": 90, "Physics": 70, "English": 80}, "entrance": 92, "dob": "2006-05-01"},
        {"name": "Chetan", "course": "CSE", "marks": {"Maths": 85, "Physics": 85, "English": 70}, "entrance": 80, "dob": "2006-01-20"},
        {"name": "Deepa",  "course": "CSE", "marks": {"Maths": 90, "Physics": 80, "English": 70}, "entrance": 88, "dob": "2005-12-15"},
        {"name": "Eswar",  "course": "CSE", "marks": {"Maths": 55, "Physics": 95, "English": 90}, "entrance": 90, "dob": "2006-02-02"},
        {"name": "Farah",  "course": "CSE", "marks": {"Maths": 65, "Physics": 60, "English": 70}, "entrance": 75, "dob": "2006-07-07"},
        {"name": "Gopal",  "course": "CSE", "marks": {"Maths": 95, "English": 90},                "entrance": 85, "dob": "2006-04-04"},

        {"name": "Hema",   "course": "PHY", "marks": {"Maths": 70, "Physics": 80, "Chemistry": 75}, "entrance": 70, "dob": "2006-06-06"},
        {"name": "Imran",  "course": "PHY", "marks": {"Maths": 80, "Physics": 70, "Chemistry": 75}, "entrance": 65, "dob": "2006-08-08"},
        {"name": "Jaya",   "course": "PHY", "marks": {"Maths": 60, "Physics": 55, "Chemistry": 70}, "entrance": 60, "dob": "2006-09-09"},

        {"name": "Kavya",  "course": "COM", "marks": {"Accountancy": 80, "English": 60, "Economics": 70}, "entrance": 72, "dob": "2006-02-14"},
        {"name": "Lokesh", "course": "COM", "marks": {"Accountancy": 45, "English": 90, "Economics": 90}, "entrance": 80, "dob": "2006-03-03"},
        {"name": "Meena",  "course": "COM", "marks": {"Accountancy": 55, "English": 50, "Economics": 50}, "entrance": 50, "dob": "2006-10-10"},

        {"name": "Naveen", "course": "MBA", "marks": {"Maths": 90, "English": 90}, "entrance": 80, "dob": "2006-01-01"},
    ]
    print_report(candidates)
```

Test Cases

#	Candidate	Course	Key input	Expected result	Rule tested

1	Eswar	CSE	Maths 55 (needs 60), aggregate 80	Rejected: Maths 55 < 60	Subject minimum
2	Gopal	CSE	Physics not studied	Rejected: subject missing	Required subject
3	Farah	CSE	Aggregate 65 (cutoff 70)	Rejected: aggregate	Course cutoff
4	Jaya	PHY	Physics 55 (needs 60)	Rejected	Different course rules
5	Lokesh	COM	Accountancy 45, aggregate 75	Rejected: subject	Good aggregate cannot save a failed subject
6	Meena	COM	Aggregate 51.67 (cutoff 55)	Rejected: aggregate	Cutoff per course
7	Naveen	MBA	Course not in list	Invalid application	Unknown course
8	Arun, Bhavna, Chetan, Deepa	CSE	All aggregate 80	Order decided by tie-breaks	Tie resolution
9	Bhavna vs Arun	CSE	Same aggregate and Maths; entrance 92 vs 88	Bhavna first	Tie-break 3 (entrance)
10	Deepa vs Arun	CSE	Same aggregate, Maths, entrance; DOB 2005-12-15 vs 2006-03-10	Deepa first	Tie-break 4 (older)
11	Chetan	CSE	Same aggregate, Maths 85 (lower)	Rank 4, waitlisted	Tie-break 2 (Maths) and seat limit
12	Kavya	COM	Only one eligible for 2 seats	Admitted, no waitlist	Fewer eligible than seats

Output

```
================================================================================
COURSE: CSE   Seats: 3   Cutoff: 70.0%   Tie-break subject: Maths
================================================================================
Rank Name     Agg%    Maths    Entrance  DOB         Status
--------------------------------------------------------------------------------
1    Bhavna   80.00   90       92        2006-05-01  ADMITTED
2    Deepa    80.00   90       88        2005-12-15  ADMITTED
3    Arun     80.00   90       88        2006-03-10  ADMITTED
4    Chetan   80.00   85       80        2006-01-20  WAITLIST

Rejected:
  Eswar    -> Maths 55 < 60
  Farah    -> Aggregate 65.00% < cutoff 70.0%
  Gopal    -> Required subject Physics not studied

================================================================================
COURSE: PHY   Seats: 2   Cutoff: 60.0%   Tie-break subject: Physics
================================================================================
Rank Name     Agg%    Physics  Entrance  DOB         Status
--------------------------------------------------------------------------------
1    Hema     75.00   80       70        2006-06-06  ADMITTED
2    Imran    75.00   70       65        2006-08-08  ADMITTED
```

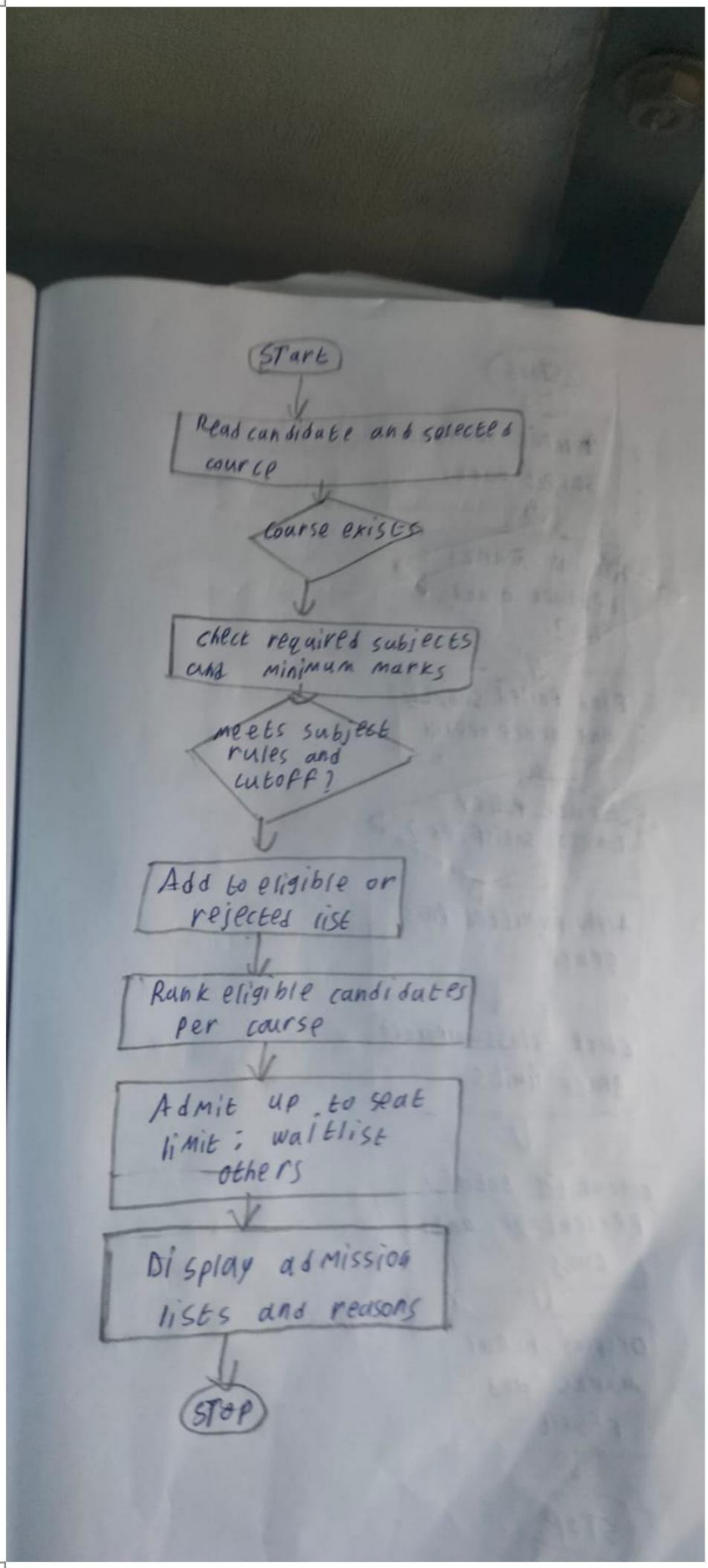
```
Rejected:
  Jaya     -> Physics 55 < 60

================================================================================
COURSE: COM   Seats: 2   Cutoff: 55.0%   Tie-break subject: Accountancy
================================================================================
Rank Name     Agg%    Account  Entrance  DOB         Status
--------------------------------------------------------------------------------
1    Kavya    70.00   80       72        2006-02-14  ADMITTED

Rejected:
  Lokesh   -> Accountancy 45 < 50
  Meena    -> Aggregate 51.67% < cutoff 55.0%

Invalid application: Naveen -> Unknown course 'MBA'
```

Flowchart


START
Read candidate and selected course
Course exists
check required subjects and minimum marks
meets subject rules and cutoff?
Add to eligible or rejected list
Rank eligible candidates per course
Admit up to seat limit; waitlist others
Display admission lists and reasons
STOP

Conclusion

The engine applies each course's own rules in a fixed order: subject eligibility first, then the aggregate cutoff. It then ranks the survivors and resolves ties through a clear chain of criteria. Because every rejection comes with a reason and every tie is settled by a documented rule, the process is transparent and repeatable. All course rules live in one COURSES dictionary, so a new course can be added by adding one entry, without changing any logic.

Problem Statement

After an exam, each student has marks in several subjects. A student passes only if they score the **minimum in every subject** . Some students miss the pass mark by just a few marks, so the university allows **grace marks** under strict limits. The system must apply the pass rule and the grace policy, make sure **no subject goes above the maximum marks**, and give every student a final **result and class** .

Objectives

Check that the marks entered are valid (0 to 100).

Apply the **subject-wise pass rule** (at least 40 in every subject).

Apply the **grace-mark policy** only where it is allowed.

Make sure grace never pushes a subject above 100 or breaks a limit.

Calculate the total, percentage and **class** (Distinction, First, Second, Pass Class or Fail).

Assumptions

Rule	Value
A failed subject is raised to 40 only if it is short by	at most **5** marks
Grace to pass is given only if the student failed	at most **2** subjects
Maximum grace per subject (both kinds together)	**5**
Maximum grace per student (both kinds together)	**10**
Class grace: a student who has passed everything and is short of the next class by at most **5** marks may get grace to reach it	if rules 3 and 4 still hold
No subject can go above	**100**

Algorithm

1. Read the student's marks.
2. **Validate:** if any mark is below 0 or above 100, the result is INVALID.
3. **Pass grace:**
    a. List the subjects with marks below 40.
    b. If more than 2 are failed → FAIL.
    c. If any is short by more than 5 → FAIL.
    d. If the total grace needed is more than 10 → FAIL.

e. Otherwise raise each failed subject to exactly 40.

4. **Class grace:**

   a. Find the total after pass grace and the next class boundary above it.

   b. Shortfall = boundary marks − total.

   c. If shortfall ≤ 5 and total grace + shortfall ≤ 10, try the lowest-scoring subject first.

   d. Give the grace to the first subject where the mark stays ≤ 100 and its grace stays ≤ 5.

1. Final total = original + grace. Calculate the percentage and the class.
2. Print the result.

Pseudocode

```
CONSTANTS: MAX=100, PASS=40, PASS_GRACE_LIMIT=5, MAX_FAILED=2,
        MAX_TOTAL_GRACE=10, MAX_SUBJECT_GRACE=5, CLASS_GRACE_LIMIT=5

FUNCTION give_pass_grace(marks):
   failed = subjects with mark < PASS
   IF count(failed) > MAX_FAILED            RETURN None   (fail)
   FOR each failed subject:
      need = PASS - mark
      IF need > PASS_GRACE_LIMIT            RETURN None
      grace[subject] = need
   IF sum(grace) > MAX_TOTAL_GRACE          RETURN None
   RETURN grace

FUNCTION give_class_grace(marks, grace):
   total = sum(marks) + sum(grace)
   boundary = next class boundary above total
   IF no boundary                       RETURN grace   (already Distinction)
   shortfall = boundary - total
   IF shortfall > CLASS_GRACE_LIMIT          RETURN grace
   IF sum(grace) + shortfall > MAX_TOTAL_GRACE RETURN grace
   FOR each subject from lowest marks to highest:
      IF mark + grace + shortfall <= MAX AND grace + shortfall <=
MAX_SUBJECT_GRACE:
         grace[subject] += shortfall
         RETURN grace
   RETURN grace

MAIN:
   FOR each student:
      IF marks invalid:  PRINT INVALID; CONTINUE
      grace = give_pass_grace(marks)
```

IF grace is None: PRINT FAIL; CONTINUE
grace = give_class_grace(marks, grace)
final_total = sum(marks) + sum(grace)
percent = final_total * 100 / (MAX * number_of_subjects)
PRINT result and class(percent)

Python source code

```
# Examination Result and Grace-Mark Validation

MAX_MARKS = 100
PASS_MARK = 40
PASS_GRACE_LIMIT = 5
MAX_FAILED = 2
MAX_TOTAL_GRACE = 10
MAX_SUBJECT_GRACE = 5
CLASS_GRACE_LIMIT = 5


CLASSES = [(75, "Distinction"), (60, "First Class"), (50, "Second Class")]


def is_valid(marks):
    """All marks must be whole numbers from 0 to 100."""
    for mark in marks.values():
        if not isinstance(mark, int) or mark < 0 or mark > MAX_MARKS:
            return False
    return True


def get_class(percent):
    for limit, name in CLASSES:
        if percent >= limit:
            return name
    return "Pass Class"
```

```python
def give_pass_grace(marks):
    """Step 1: raise failed subjects to 40. Returns grace dict, or None if not allowed."""
    grace = {subject: 0 for subject in marks}
    failed = [s for s in marks if marks[s] < PASS_MARK]

    if len(failed) > MAX_FAILED:
        return None
    for s in failed:
        need = PASS_MARK - marks[s]
        if need > PASS_GRACE_LIMIT:
            return None
        grace[s] = need
    if sum(grace.values()) > MAX_TOTAL_GRACE:
        return None
    return grace


def give_class_grace(marks, grace):
    """Step 2: if a few marks short of the next class, add grace to ONE subject."""
    total = sum(marks.values()) + sum(grace.values())
    max_total = MAX_MARKS * len(marks)


    for limit, name in reversed(CLASSES):
        boundary = limit * max_total // 100
        if total < boundary:
            shortfall = boundary - total
            break
    else:
        return grace
```

```python
    if shortfall > CLASS_GRACE_LIMIT:
        return grace
    if sum(grace.values()) + shortfall > MAX_TOTAL_GRACE:
        return grace


    for s in sorted(marks, key=lambda x: marks[x] + grace[x]):
        new_mark = marks[s] + grace[s] + shortfall
        new_grace = grace[s] + shortfall
        if new_mark <= MAX_MARKS and new_grace <= MAX_SUBJECT_GRACE:
            grace[s] = new_grace
            return grace
    return grace


def process(name, marks):
    if not is_valid(marks):
        return [name, "-", "-", "-", "-", "INVALID", "-"]

    grace = give_pass_grace(marks)
    if grace is None:
        total = sum(marks.values())
        return [name, total, 0, total, f"{total * 100 / (MAX_MARKS * len(marks)):.1f}", "FAIL", "-"]

    grace = give_class_grace(marks, grace)
    original = sum(marks.values())
    used = sum(grace.values())
    final = original + used
    percent = final * 100 / (MAX_MARKS * len(marks))
    return [name, original, used, final, f"{percent:.1f}", "PASS", get_class(percent)]
```

```python
if __name__ == "__main__":
    subjects = ["Maths", "Physics", "Chemistry", "English", "CompSci"]
    data = [
        ("Priya",  [80, 85, 78, 90, 72]),
        ("Suma",   [36, 64, 64, 64, 65]),
        ("Tarun",  [38, 39, 60, 70, 80]),
        ("Uma",    [35, 35, 35, 70, 70]),
        ("Vikram", [30, 80, 80, 80, 80]),
        ("Wasim",  [100, 100, 70, 60, 43]),
        ("Meera",  [36, 36, 72, 72, 73]),
        ("Yash",   [100, 100, 100, 100, 100]),
        ("Arjun",  [105, 70, 70, 70, 70]),
    ]

    print("EXAMINATION RESULTS (max 500 marks, pass mark 40 per subject)")
    print("-" * 66)
    print(f"{'Name':<8}{'Original':<10}{'Grace':<7}{'Final':<7}{'%':<7}{'Result':<9}Class")
    print("-" * 66)
    for name, m in data:
        row = process(name, dict(zip(subjects, m)))
        print(f"{row[0]:<8}{row[1]:<10}{row[2]:<7}{row[3]:<7}{row[4]:<7}{row[5]:<9}{row[6]}")
```

Test Cases

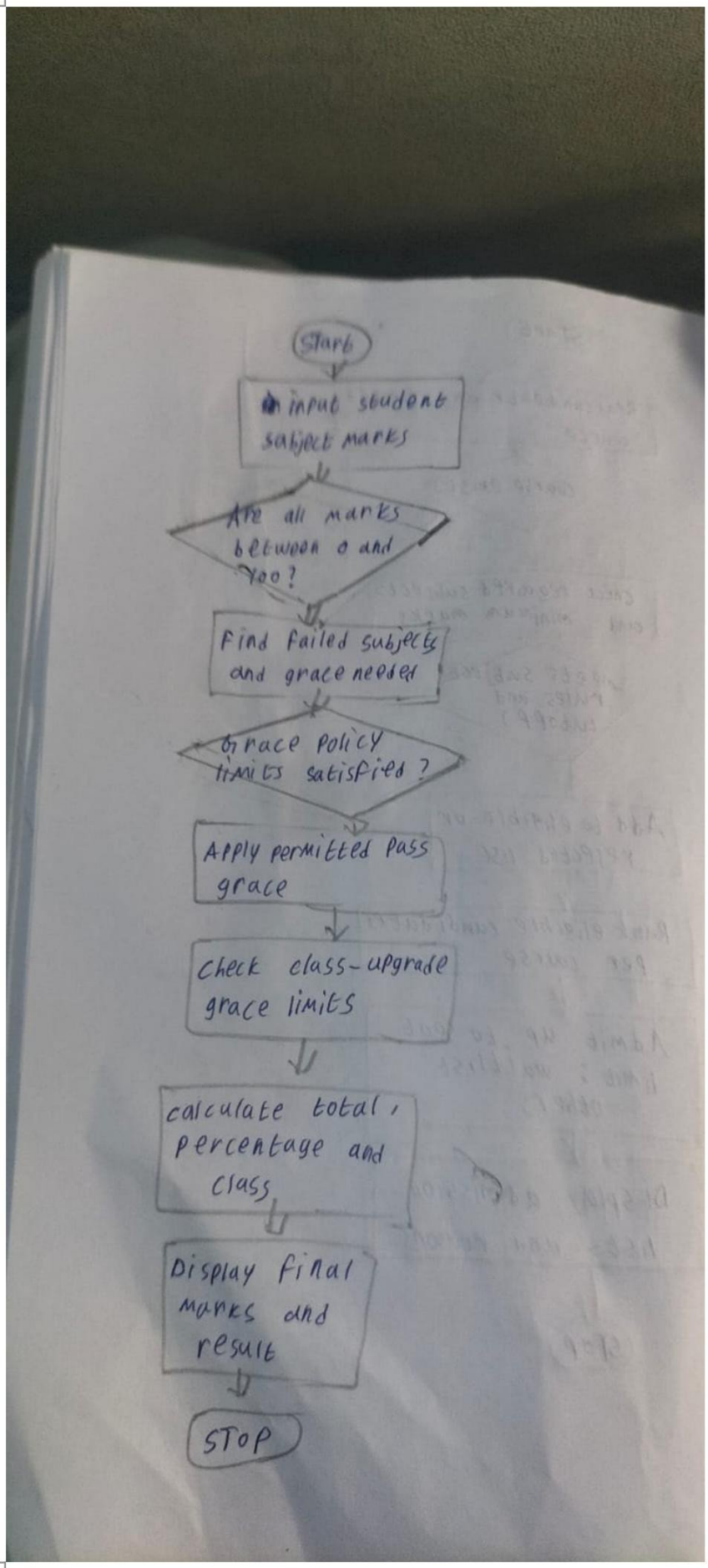
Student	Marks	Expected result	What it tests
Priya	80, 85, 78, 90, 72	PASS, 405, Distinction, no grace	Normal pass
Suma	36, 64, 64, 64, 65	Grace 7, 300, First Class	Pass grace (+4) then class grace (+3) given to a different subject
Tarun	38, 39, 60, 70, 80	Grace 3, 290, Second Class	Two failed subjects, both graced
Uma	35, 35, 35, 70, 70	FAIL	Three failures: no grace
Vikram	30, 80, 80, 80, 80	FAIL	Short by 10 (more than 5)
Wasim	100, 100, 70, 60, 43	Grace 2, 375 (75%), Distinction	Class grace, exact boundary
Meera	36, 36, 72, 72, 73	Grace 8, 297, Second Class	Total grace limit (8 + 3 > 10) blocks the upgrade

Yash	100 in all five	PASS, 500, Distinction	Nothing goes above 100
Arjun	105, 70, 70, 70, 70	INVALID	Mark above maximum

Output

```
EXAMINATION RESULTS (max 500 marks, pass mark 40 per subject)
------------------------------------------------------------------
Name    Original  Grace  Final  %      Result   Class
------------------------------------------------------------------
Priya   405       0      405    81.0   PASS     Distinction
Suma    293       7      300    60.0   PASS     First Class
Tarun   287       3      290    58.0   PASS     Second Class
Uma     245       0      245    49.0   FAIL     -
Vikram  350       0      350    70.0   FAIL     -
Wasim   373       2      375    75.0   PASS     Distinction
Meera   289       8      297    59.4   PASS     Second Class
Yash    500       0      500    100.0  PASS     Distinction
Arjun   -         -      -      -      INVALID  -
```

Flowchart


Start
input student subject marks
Are all marks between 0 and 100?
Find failed subjects and grace needed
Grace policy limits satisfied?
Apply permitted pass grace
Check class-upgrade grace limits
calculate total, percentage and class
Display final marks and result
STOP

Conclusion

The program follows a fixed order: validate the marks, give pass grace, give class grace, then classify. Every rule is a constant at the top of the file, so no student can receive more grace than the policy allows and no subject can go above 100. Because the original marks, grace and final total are shown side by side, every result can be checked by hand. To change the policy (for example a different pass mark or grace limit), you only need to edit the constants.

Problem Statement

A college timetable lists many classes. Each class has a course name, a day, a start time, an end time and a room. If two classes overlap in time, or two classes need the same room at the same time, the timetable has a **conflict**. Checking this by hand is slow and error-prone.

Objectives

Represent a class schedule using tuples and strings.

Convert time strings to numbers so they can be compared.

Detect overlapping periods on the same day.

Find room conflicts (same room, overlapping time).

List rooms that are unavailable in a given slot.

Validate a timetable and report clear error messages.

Assumptions

Each class is a tuple: (course, day, start, end, room).

Time is in 24-hour "HH:MM" format, e.g. "14:30".

Valid days are Mon to Sat.

A class lies within one day (no class crosses midnight).

Back-to-back classes (one ends 10:00, next starts 10:00) are **not** a conflict.

Two classes in **different rooms** at the same time are allowed, and the timetable is invalid only if the same room is double-booked.

The list of valid rooms is given.

Algorithm

1. Convert each "HH:MM" string into minutes since midnight (09:30 becomes 570).
2. Two periods overlap if start1 < end2 **and** start2 < end1.
3. **Find overlaps:** compare every pair of classes. If the days are the same and the periods overlap, store the pair.
4. **Room conflicts:** from the overlapping pairs, keep those with the same room.
5. **Unavailable rooms:** for a requested day and time, collect the rooms of all classes whose period overlaps it.
6. **Validate:** for each class check the format, day, start < end, and room exists. If there are no errors so far, also check for room conflicts. Valid means the

error list is empty.

Pseudocode

```
FUNCTION time_to_minutes(t):
  split t into hours and minutes
  RETURN hours * 60 + minutes

FUNCTION is_overlap(s1, e1, s2, e2):
  RETURN s1 < e2 AND s2 < e1

FUNCTION find_overlaps(timetable):
  result = empty list
  FOR i = 0 TO n-1:
    FOR j = i+1 TO n-1:
      IF day[i] == day[j] AND is_overlap(times of i, times of j):
        add (class i, class j) to result
  RETURN result

FUNCTION find_room_conflicts(timetable):
  RETURN pairs from find_overlaps WHERE room1 == room2

FUNCTION unavailable_rooms(timetable, day, start, end):
  busy = empty set
  FOR each class in timetable:
    IF class.day == day AND is_overlap(start, end, class.start, class.end):
      add class.room to busy
  RETURN busy

FUNCTION validate_timetable(timetable, rooms):
  errors = empty list
  FOR each class:
    check format, day, start < end, room in rooms
    add a message to errors for every problem
  IF errors is empty:
    FOR each room conflict: add message to errors
  RETURN (errors is empty, errors)
```

Python source code

```python
# Student Timetable Conflict Detection

VALID_DAYS = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]


def time_to_minutes(t):
    """Convert 'HH:MM' string to minutes since midnight."""
    hours, minutes = t.split(":")
    return int(hours) * 60 + int(minutes)


def is_overlap(start1, end1, start2, end2):
    """Two periods overlap if each starts before the other ends."""
    s1, e1 = time_to_minutes(start1), time_to_minutes(end1)
    s2, e2 = time_to_minutes(start2), time_to_minutes(end2)
    return s1 < e2 and s2 < e1


def find_overlaps(timetable):
    """Return every pair of classes that overlap in time on the same day."""
    result = []
    for i in range(len(timetable)):
        for j in range(i + 1, len(timetable)):
            a, b = timetable[i], timetable[j]
            if a[1] == b[1] and is_overlap(a[2], a[3], b[2], b[3]):
                result.append((a, b))
    return result


def find_room_conflicts(timetable):
    """Overlapping classes that also use the same room."""
    return [(a, b) for a, b in find_overlaps(timetable) if a[4] == b[4]]
```

```python
def unavailable_rooms(timetable, day, start, end):
    """Set of rooms that are busy during the requested period."""
    busy = set()
    for course, d, s, e, room in timetable:
        if d == day and is_overlap(start, end, s, e):
            busy.add(room)
    return busy


def validate_timetable(timetable, rooms):
    """Check a proposed timetable. Returns (is_valid, list_of_errors)."""
    errors = []
    for entry in timetable:
        if len(entry) != 5:
            errors.append(f"Bad format: {entry}")
            continue
        course, day, start, end, room = entry
        if day not in VALID_DAYS:
            errors.append(f"{course}: invalid day '{day}'")
        try:
            if time_to_minutes(start) >= time_to_minutes(end):
                errors.append(f"{course}: start time must be before end time")
        except ValueError:
            errors.append(f"{course}: invalid time format")
        if room not in rooms:
            errors.append(f"{course}: unknown room '{room}'")
    if not errors:
        for a, b in find_room_conflicts(timetable):
            errors.append(f"Room clash: {a[0]} and {b[0]} in {a[4]} on {a[1]}")
    return len(errors) == 0, errors
```

```python
if __name__ == "__main__":
    rooms = ["R101", "R102", "Lab1"]
    timetable = [
        ("Maths",   "Mon", "09:00", "10:00", "R101"),
        ("Physics", "Mon", "09:30", "10:30", "R101"),
        ("Chem",    "Mon", "09:45", "11:00", "Lab1"),
        ("English", "Tue", "10:00", "11:00", "R102"),
    ]
    print("Overlapping classes:")
    for a, b in find_overlaps(timetable):
        print(" ", a[0], "<->", b[0])
    print("Room conflicts:")
    for a, b in find_room_conflicts(timetable):
        print(" ", a[0], "<->", b[0], "in", a[4])
    print("Busy rooms Mon 09:00-10:00:", sorted(unavailable_rooms(timetable, "Mon", "09:00", "10:00")))
    ok, errs = validate_timetable(timetable, rooms)
    print("Valid?", ok)
    for e in errs:
        print(" -", e)
```

Test Cases

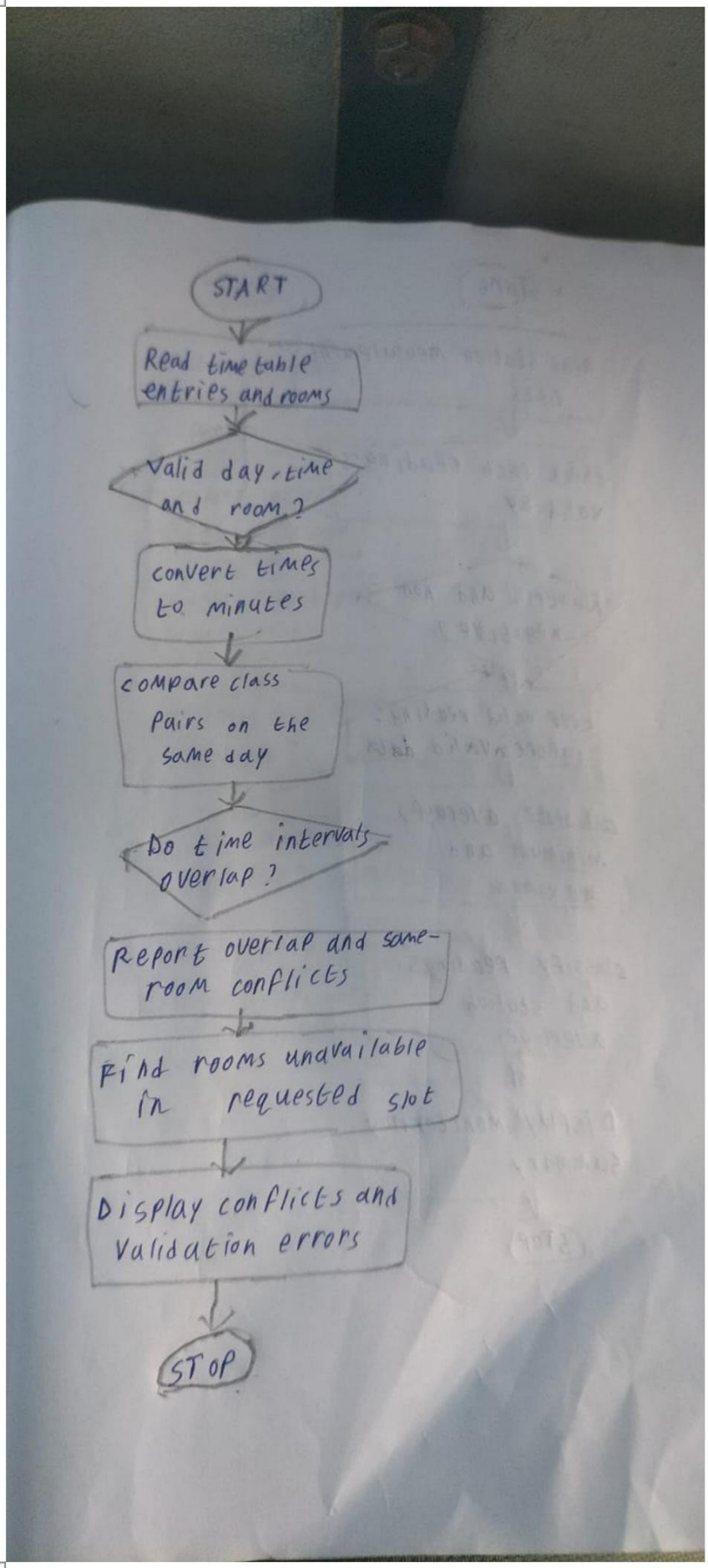
#	Test	Input	Expected result
1	Partial overlap	09:00-10:00 vs 09:30-11:00	Overlap = True

2	Back-to-back classes	09:00-10:00 vs 10:00-11:00	Overlap = False
4	Same time, different days	Mon 09:00 vs Tue 09:00	No overlap
5	Same time, same room	Maths & Physics in R101	Room conflict found
6	Same time, different rooms	Maths (R101) & Chem (Lab1)	Overlap, but no room conflict
7	Busy rooms Mon 09:00-10:00	Sample timetable	['Lab1', 'R101']
8	Free slot	Mon 11:00-12:00	Empty set
9	Valid timetable	Maths 9-10, Physics 10-11 in R101	(True, [])
10	Bad data	("X","Sun","10:00","09:00","R999")	Invalid day, bad time order, unknown room

Output

```
Overlapping classes:
  Maths <-> Physics
  Maths <-> Chem
  Physics <-> Chem
Room conflicts:
  Maths <-> Physics in R101
Busy rooms Mon 09:00-10:00: ['Lab1', 'R101']
Valid? False
 - Room clash: Maths and Physics in R101 on Mon
```

Flowchart


START
Read time table entries and rooms
Valid day, time and room?
convert times to minutes
compare class pairs on the same day
Do time intervals overlap?
Report overlap and same-room conflicts
Find rooms unavailable in requested slot
Display conflicts and validation errors
STOP

Conclusion

The program takes a timetable stored as tuples and strings and finds overlapping periods, room clashes and busy rooms, then validates the whole timetable with clear error messages. Converting times to minutes makes the comparison simple, and the rule start1 < end2 and start2 < end1 correctly handles partial overlaps and back-to-back classes.

Problem Statement

Environmental agencies collect air-quality or water-quality readings from many monitoring stations (for example PM2.5 values or pollutant levels). The readings are stored as **nested lists**, where each station has its own list of measurements. Checking them by hand is slow, and bad values (sensor errors, negative numbers) can spoil the results.

Objectives

Store station readings in a nested list.

Clean the data by ignoring invalid readings.

Calculate average, minimum and maximum for each station.

Classify each reading and each station's average using thresholds.

Display a clear report for every station.

Assumptions

Each station is stored as [station_name, [reading1, reading2, ...]].

A valid reading is a number that is zero or greater. Text and negative values are treated as sensor errors and skipped.

Thresholds are supplied as a list of (upper_limit, label) pairs, in increasing order.

A reading belongs to the first category whose limit it does not exceed. With the limits used here, 50 is Good and 51 is Moderate.

The threshold values used (50, 100, 150) are example values, and you can change them for any pollutant.

A station with no valid readings is reported as "No valid readings".

Algorithm

1. Define the threshold list.
2. For each station in the nested list, take its name and readings.
3. **Clean:** keep only numeric readings that are zero or greater, and count how many were skipped.
4. If no valid readings remain, report "No data" and move on to the next station.
5. **Average:** sum of valid readings divided by their count.
6. **Extremes:** find the minimum and maximum of the valid readings.
7. **Classify:** compare a value with each threshold limit in order, and return the first label where value <= limit.
8. Classify every reading and also the station

Pseudocode

```
thresholds = [(50,"Good"), (100,"Moderate"), (150,"Unhealthy"), (infinity,"Hazardous")]

FUNCTION clean_readings(readings):
    valid = empty list
    FOR each r in readings:
        IF r is a number AND r >= 0:
            add r to valid
    RETURN valid

FUNCTION calculate_average(values):
    IF values is empty: RETURN None
    RETURN sum(values) / count(values)

FUNCTION find_extremes(values):
    IF values is empty: RETURN None, None
    RETURN min(values), max(values)

FUNCTION classify(value, thresholds):
    FOR each (limit, label) in thresholds:
        IF value <= limit: RETURN label

FUNCTION analyze_station(name, readings):
    valid = clean_readings(readings)
    average = calculate_average(valid)
    low, high = find_extremes(valid)
    labels = classify each value in valid
    RETURN results

MAIN:
    FOR each [name, readings] in data:
        results = analyze_station(name, readings)
        print results
```

Python source code

```python
# Environmental Monitoring Data Analyzer

THRESHOLDS = [
    (50, "Good"),
    (100, "Moderate"),
    (150, "Unhealthy"),
    (float("inf"), "Hazardous"),
]


def clean_readings(readings):
    """Keep only valid numbers (not text, not negative)."""
    valid = []
    for r in readings:
        if isinstance(r, (int, float)) and r >= 0:
            valid.append(r)
    return valid


def calculate_average(values):
    """Average of a list; returns None for an empty list."""
    if len(values) == 0:
        return None
    return sum(values) / len(values)


def find_extremes(values):
    """Return (minimum, maximum); (None, None) for an empty list."""
    if len(values) == 0:
        return None, None
    return min(values), max(values)
```

```python
def classify(value, thresholds):
    """Return the category label for one reading."""
    for limit, label in thresholds:
        if value <= limit:
            return label
    return "Unknown"


def analyze_station(name, readings, thresholds):
    """Analyse one station and return a dictionary of results."""
    valid = clean_readings(readings)
    low, high = find_extremes(valid)
    avg = calculate_average(valid)
    result = {
        "station": name,
        "count": len(valid),
        "skipped": len(readings) - len(valid),
        "average": avg,
        "min": low,
        "max": high,
        "avg_class": classify(avg, thresholds) if avg is not None else "No data",
        "labels": [classify(v, thresholds) for v in valid],
    }
    return result


def analyze_all(data, thresholds):
    return [analyze_station(name, readings, thresholds) for name, readings in data]
```

```python
if __name__ == "__main__":
    data = [
        ["Station A", [35, 42, 58, 47]],
        ["Station B", [90, 120, 160, 75]],
        ["Station C", [20, "error", -5, 30]],
        ["Station D", []],
    ]
    for r in analyze_all(data, THRESHOLDS):
        print("Station:", r["station"])
        if r["count"] == 0:
            print("  No valid readings")
            continue
        print(f"  Valid readings: {r['count']} (skipped {r['skipped']})")
        print(f"  Average: {r['average']:.2f} -> {r['avg_class']}")
        print(f"  Min: {r['min']}  Max: {r['max']}")
        print("  Categories:", r["labels"])
```

Test Cases

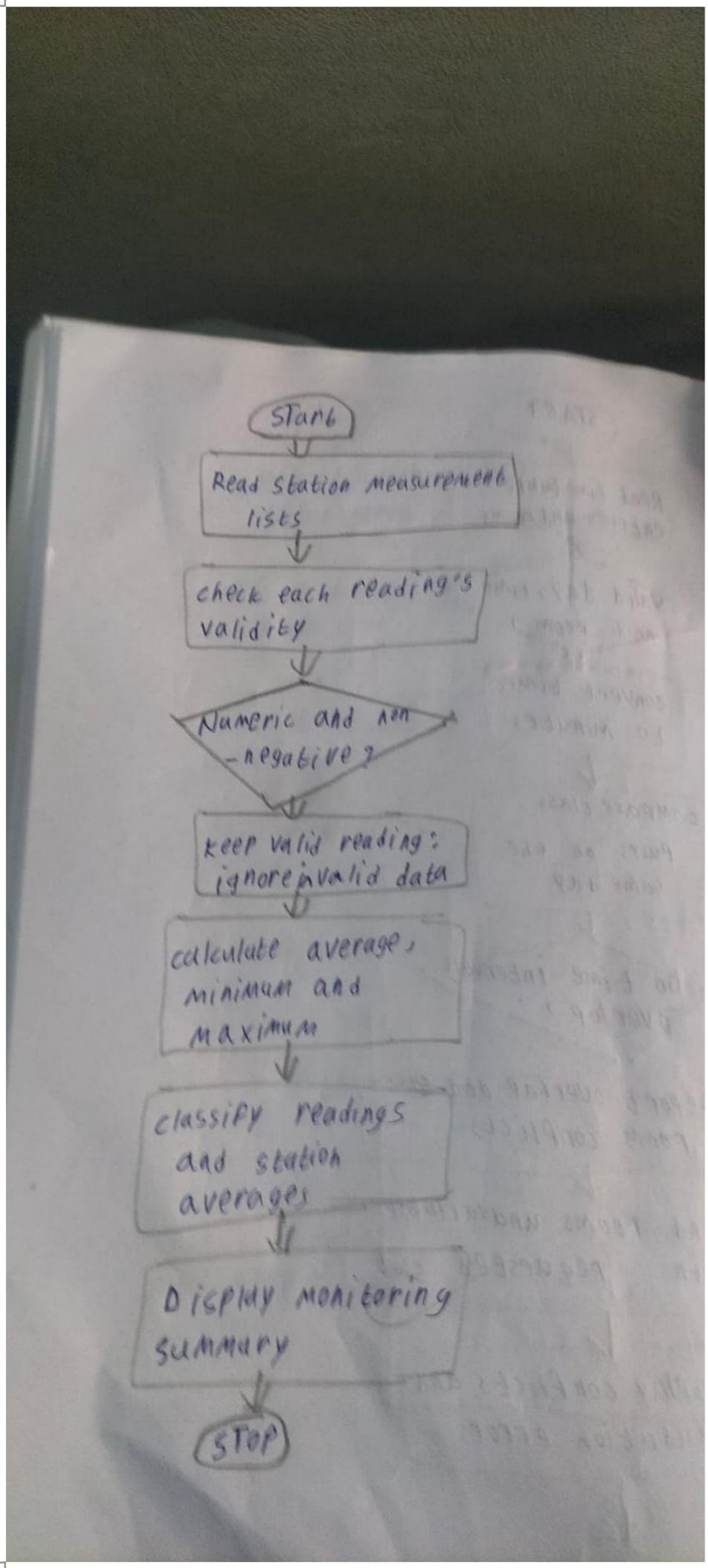
#	Test	Input	Expected result
1	Average	[10, 20, 30]	20.0
2	Empty list average	[]	None

3	Extremes	[5, 1, 9]	(1, 9)
4	Empty list extremes	[]	(None, None)
5	Boundary: 50	classify(50)	Good
6	Boundary: 51	classify(51)	Moderate
7	Boundary: 150	classify(150)	Unhealthy
8	Very high value	classify(500)	Hazardous
9	Cleaning	[10, "x", -3, 0, 7.5]	[10, 0, 7.5]
10	Station with bad data	[20, "error", -5, 30]	2 valid, 2 skipped, average 25.0
11	Station with no readings	[]	“No valid readings”

Output

```
Station: Station A
  Valid readings: 4 (skipped 0)
  Average: 45.50 -> Good
  Min: 35  Max: 58
  Categories: ['Good', 'Good', 'Moderate', 'Good']
Station: Station B
  Valid readings: 4 (skipped 0)
  Average: 111.25 -> Unhealthy
  Min: 75  Max: 160
  Categories: ['Moderate', 'Unhealthy', 'Hazardous', 'Moderate']
Station: Station C
  Valid readings: 2 (skipped 2)
  Average: 25.00 -> Good
  Min: 20  Max: 30
  Categories: ['Good', 'Good']
Station: Station D
  No valid readings
```

Flowchart


Start
Read station measurement lists
check each reading's validity
Numeric and non-negative?
keep valid reading: ignore invalid data
calculate average, minimum and maximum
classify readings and station averages
Display monitoring summary
STOP

Conclusion

The program analyses nested-list readings for several stations. It cleans invalid data, calculates averages and extremes, and classifies each reading and each station's average against the supplied thresholds. Because the thresholds are passed in as data, the same code works for air quality, water quality or any other measurement, and you only need to change the threshold list.

Problem Statement

Mobile operators offer many prepaid plans, each with a different price, validity, data allowance and call minutes. If a user goes over a plan's limits, **excess charges** are added. A plan that looks cheap can turn out costly once those extra charges are included, and a plan may not last as long as the user needs.

Objectives

Store plans as tuples and the user's needs as a dictionary.

Calculate excess data and call charges for each plan.

Check eligibility (validity and budget) for each plan.

Compare the total cost (price + excess) of all eligible plans.

Display a comparison table and recommend the best plan.

Assumptions

Each plan is a tuple: (name, price, validity_days, data_gb, call_minutes, extra_per_gb, extra_per_min).

Prices are in rupees (Rs).

Usage is the user's total data (GB) and call minutes over the period they need, plus the number of days the plan must last.

Total cost = plan price + excess charges.

Excess data charge = (usage − plan data) × rate per GB, if usage is more than the plan's data. Call minutes work the same way.

A plan is **eligible** only if its validity is at least the days needed **and** its total cost is within the budget.

If two plans have the same total cost, the one with the lower base price wins.

SMS and taxes are ignored to keep things simple.

Algorithm

1. Define the list of plans, the user's usage and the budget.
2. For each plan, calculate the excess data and excess minutes (zero if within limits).
3. Excess charge = extra data × rate per GB + extra minutes × rate per minute.
4. Total cost = price + excess charge.
5. Check validity. If it is shorter than the days needed, mark the plan "validity too short".
6. Check budget. If the total cost is more than the budget, mark it "over budget".

7. Otherwise mark it “eligible”.
8. Among eligible plans, pick the one with the lowest total cost.
9. If there are no eligible plans, report that no plan fits.

Pseudocode

```
FUNCTION calculate_excess(plan, usage):
   extra_data = MAX(0, usage.data - plan.data)
   extra_minutes = MAX(0, usage.minutes - plan.minutes)
   RETURN extra_data * plan.per_gb + extra_minutes * plan.per_min

FUNCTION check_eligibility(plan, usage, budget):
   total = plan.price + calculate_excess(plan, usage)
   IF plan.validity < usage.days: RETURN False, total, "validity too short"
   IF total > budget:             RETURN False, total, "over budget"
   RETURN True, total, "eligible"

FUNCTION best_plan(results):
   eligible = results that are eligible
   IF eligible is empty: RETURN None
   RETURN plan with the smallest total (ties: smaller price)

MAIN:
   FOR each plan in plans:
      evaluate the plan and store the result
   best = best_plan(results)
   print table and best plan
```

Python source code

```python
# Multi-Plan Telecommunications Cost Optimization

def calculate_excess(plan, usage):
    """Extra charge for data and call minutes used beyond the plan's limits."""
    name, price, validity, data_gb, minutes, per_gb, per_min = plan
    extra_data = max(0, usage["data_gb"] - data_gb)
    extra_minutes = max(0, usage["minutes"] - minutes)
    return extra_data * per_gb + extra_minutes * per_min

def check_eligibility(plan, usage, budget):
    """Return (eligible, total_cost, reason)."""
    name, price, validity, data_gb, minutes, per_gb, per_min = plan
    excess = calculate_excess(plan, usage)
    total = price + excess
    if validity < usage["days"]:
        return False, total, "validity too short"
    if total > budget:
        return False, total, "over budget"
    return True, total, "eligible"

def evaluate_plans(plans, usage, budget):
    """Evaluate every plan and return a list of result dictionaries."""
    results = []
    for plan in plans:
        eligible, total, reason = check_eligibility(plan, usage, budget)
        results.append({
            "name": plan[0],
            "price": plan[1],
            "excess": calculate_excess(plan, usage),
            "total": total,
            "eligible": eligible,
            "reason": reason,
        })
    return results
def best_plan(results):
    """Cheapest eligible plan (ties: lower base price). None if nothing fits."""
    eligible = [r for r in results if r["eligible"]]
    if not eligible:
        return None
    return min(eligible, key=lambda r: (r["total"], r["price"]))

if __name__ == "__main__":
    plans = [
        ("Basic",   199,  28,  28, 100, 10, 1.0),
        ("Value",   299,  28,  56, 300,  8, 0.8),
        ("Max",     499,  56, 112, 600,  5, 0.5),
        ("Monthly", 399,  30,  45, 200,  6, 0.6),
        ("Short",   149,  14,  14,  50, 12, 1.2),
    ]
    usage = {"data_gb": 40, "minutes": 150, "days": 28}
    budget = 450

    results = evaluate_plans(plans, usage, budget)
    print(f"{'Plan':<9}{'Price':>6}{'Excess':>8}{'Total':>8}  Status")
    for r in results:
        print(f"{r['name']:<9}{r['price']:>6}{r['excess']:>8.1f}{r['total']:>8.1f}  {r['reason']}")
    best = best_plan(results)
    if best:
        print(f"\nBest plan: {best['name']} (total Rs {best['total']:.1f})")
    else:
        print("\nNo plan fits the budget and validity.")
```

Test Cases

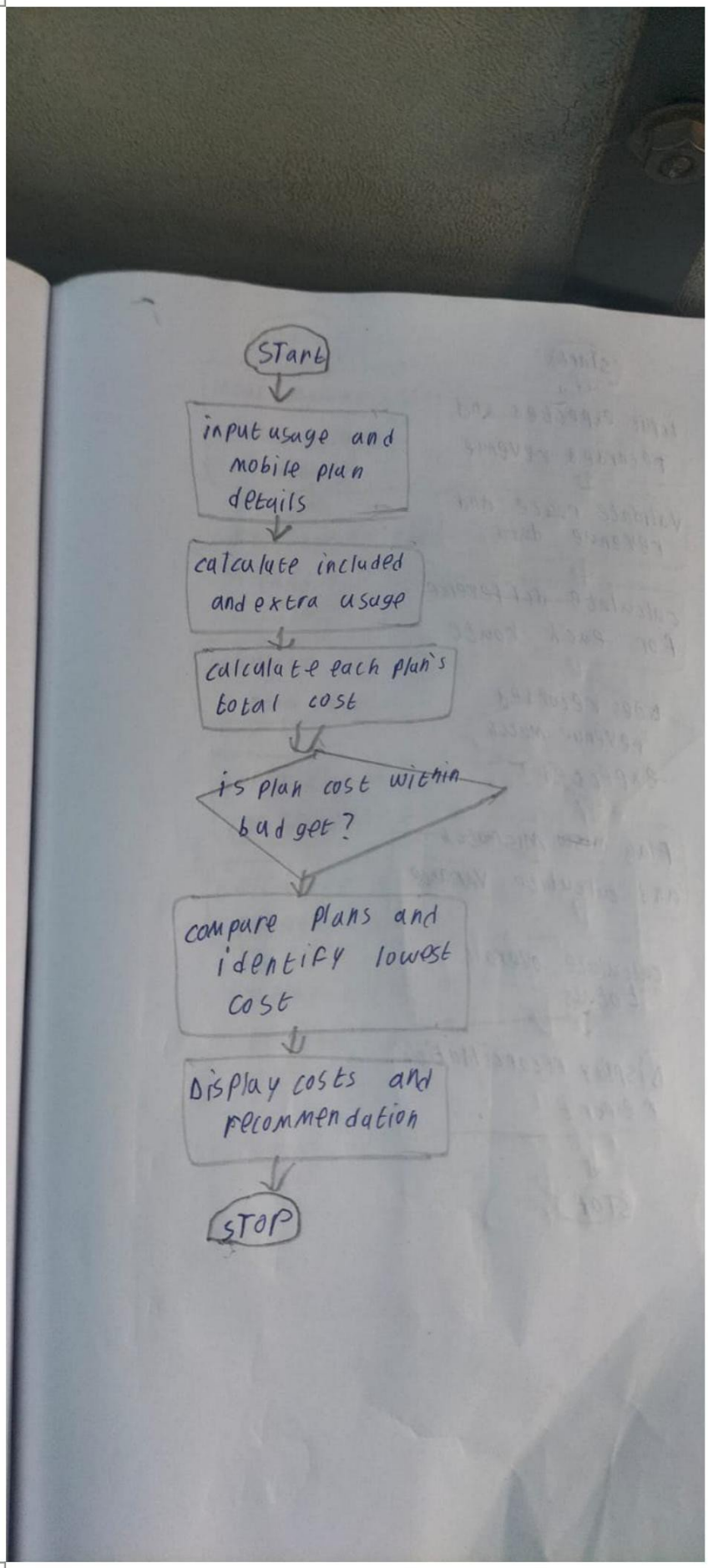
Test	Input	Expected result
No excess	Basic plan, usage 20 GB / 50 min	Excess = 0
Excess on both	Basic plan, usage 30 GB / 120 min	Excess = 2×10 + 20×1 = 40
Validity too short	Basic (28 days), need 30 days	Not eligible: validity too short
Over budget	Basic, budget 150	Not eligible: over budget
Exactly on budget	Basic, budget 199	Eligible
No plan fits	Budget 100	best_plan returns None
Excess beats low price	Plan A (Rs 300, small limits) vs Plan B (Rs 199)	Plan B chosen
Main example	Usage 40 GB / 150 min / 28 days, budget 450	Best plan = Value (Rs 299)

Output

```
Plan      Price  Excess   Total  Status
Basic       199   170.0   369.0  eligible
Value       299     0.0   299.0  eligible
Max         499     0.0   499.0  over budget
Monthly     399     0.0   399.0  eligible
Short       149   432.0   581.0  validity too short

Best plan: Value (total Rs 299.0)
```

Flowchart


START
input usage and mobile plan details
calculate included and extra usage
calculate each plan's total cost
is plan cost within budget?
compare plans and identify lowest cost
Display costs and recommendation
STOP

Conclusion

The program compares several mobile plans against the user's data, call usage, validity need and budget. It calculates excess charges, filters out plans that fail the validity or budget rules, and recommends the eligible plan with the lowest total cost. In the sample run, **Value** at Rs 299 wins. The cheapest base plan, Basic, costs Rs 369 once excess charges are added, and Max is over budget.

Problem Statement

A city bus or metro service sells tickets on several **routes** to different **passenger categories** (adult, student, senior citizen, child). Each category pays a different fare. At the end of the day, the cash and digital money collected should equal the revenue expected from the tickets issued. In practice the two often differ because of wrong fares, missing entries or errors.

Objectives

Store route fares and category discounts in dictionaries.

Store ticket records as tuples in a list.

Calculate the fare for each passenger and the expected revenue per route.

Compare expected and collected revenue for every route.

Flag each route as MATCH, SHORTAGE or EXCESS.

Detect and report invalid ticket records (unknown route or category).

Assumptions

Each ticket record is a tuple: (route, category, passenger_count).

Each route has a base fare per passenger: R1 = Rs 20, R2 = Rs 35, R3 = Rs 50.

Category fare = base fare × multiplier, where the multipliers are Adult 1.0, Student 0.5, Senior 0.6 and Child 0.0 (free). These are example values.

Collected revenue is given per route as a dictionary.

Differences of up to Rs 1 count as a match (to allow for rounding or small change errors).

Shortage means collected < expected. Excess means collected > expected.

Tickets with an unknown route or category, or a negative passenger count, are invalid and are listed separately, not counted in the revenue.

Algorithm

1. Define the base fares, category multipliers and tolerance.
2. Read the ticket list and the collected revenue.
3. For each ticket, calculate the fare per passenger (base fare × multiplier).
4. If the route or category is unknown (or the count is negative), save the ticket in the invalid list and skip it.
5. Otherwise add fare × passenger_count to that route's expected revenue.
6. For each route, calculate difference = collected − expected.
7. If |difference| <= tolerance, status is MATCH. If the difference is negative,

status is SHORTAGE. Otherwise it is EXCESS.

8. Print the reconciliation report, totals and invalid tickets.

Pseudocode

```
base_fares = {R1: 20, R2: 35, R3: 50}
discounts  = {Adult: 1.0, Student: 0.5, Senior: 0.6, Child: 0.0}
tolerance  = 1

FUNCTION calculate_fare(route, category):
    IF route not in base_fares OR category not in discounts:
        RETURN None
    RETURN base_fares[route] * discounts[category]

FUNCTION calculate_expected_revenue(tickets):
    expected = 0 for every route
    invalid = empty list
    FOR each (route, category, count) in tickets:
        fare = calculate_fare(route, category)
        IF fare is None OR count < 0:
            add ticket to invalid
        ELSE:
            expected[route] = expected[route] + fare * count
    RETURN expected, invalid

FUNCTION reconcile(expected, collected):
    FOR each route:
        diff = collected[route] - expected[route]
        IF abs(diff) <= tolerance: status = "MATCH"
        ELSE IF diff < 0:          status = "SHORTAGE"
        ELSE:                      status = "EXCESS"
        add (route, expected, collected, diff, status) to report
    RETURN report

MAIN:
    expected, invalid = calculate_expected_revenue(tickets)
    report = reconcile(expected, collected)
    print report, totals, invalid tickets
```

Python source code

```python
# Smart Public Transport Fare Reconciliation

BASE_FARES = {"R1": 20, "R2": 35, "R3": 50}
DISCOUNTS = {"Adult": 1.0, "Student": 0.5, "Senior": 0.6, "Child": 0.0}
TOLERANCE = 1

def calculate_fare(route, category):
    """Fare for ONE passenger. Returns None if route or category is unknown."""
    if route not in BASE_FARES or category not in DISCOUNTS:
        return None
    return BASE_FARES[route] * DISCOUNTS[category]

def calculate_expected_revenue(tickets):
    """Return (expected_by_route, invalid_tickets)."""
    expected = {route: 0 for route in BASE_FARES}
    invalid = []
    for route, category, count in tickets:
        fare = calculate_fare(route, category)
        if fare is None or count < 0:
            invalid.append((route, category, count))
        else:
            expected[route] += fare * count
    return expected, invalid
```

```python
def reconcile(expected, collected, tolerance=TOLERANCE):
    """Compare expected and collected revenue for every route."""
    report = []
    for route in expected:
        exp = expected[route]
        col = collected.get(route, 0)
        diff = col - exp
        if abs(diff) <= tolerance:
            status = "MATCH"
        elif diff < 0:
            status = "SHORTAGE"
        else:
            status = "EXCESS"
        report.append((route, exp, col, diff, status))
    return report

if __name__ == "__main__":
    tickets = [
        ("R1", "Adult", 10), ("R1", "Student", 6), ("R1", "Child", 3),
        ("R2", "Adult", 8),  ("R2", "Senior", 4),
        ("R3", "Adult", 5),  ("R3", "Student", 2),
        ("R9", "Adult", 2),
        ("R2", "Tourist", 1),
    ]
    collected = {"R1": 260, "R2": 400, "R3": 310}

    expected, invalid = calculate_expected_revenue(tickets)
    report = reconcile(expected, collected)
```

```python
print(f"{'Route':<7}{'Expected':>10}{'Collected':>11}{'Diff':>8}  Status")
for route, exp, col, diff, status in report:
    print(f"{route:<7}{exp:>10.1f}{col:>11.1f}{diff:>8.1f}  {status}")
print("\nTotal expected :", sum(r[1] for r in report))
print("Total collected:", sum(r[2] for r in report))
print("Net difference :", sum(r[3] for r in report))
print("Invalid tickets:", invalid)
```

Test Cases

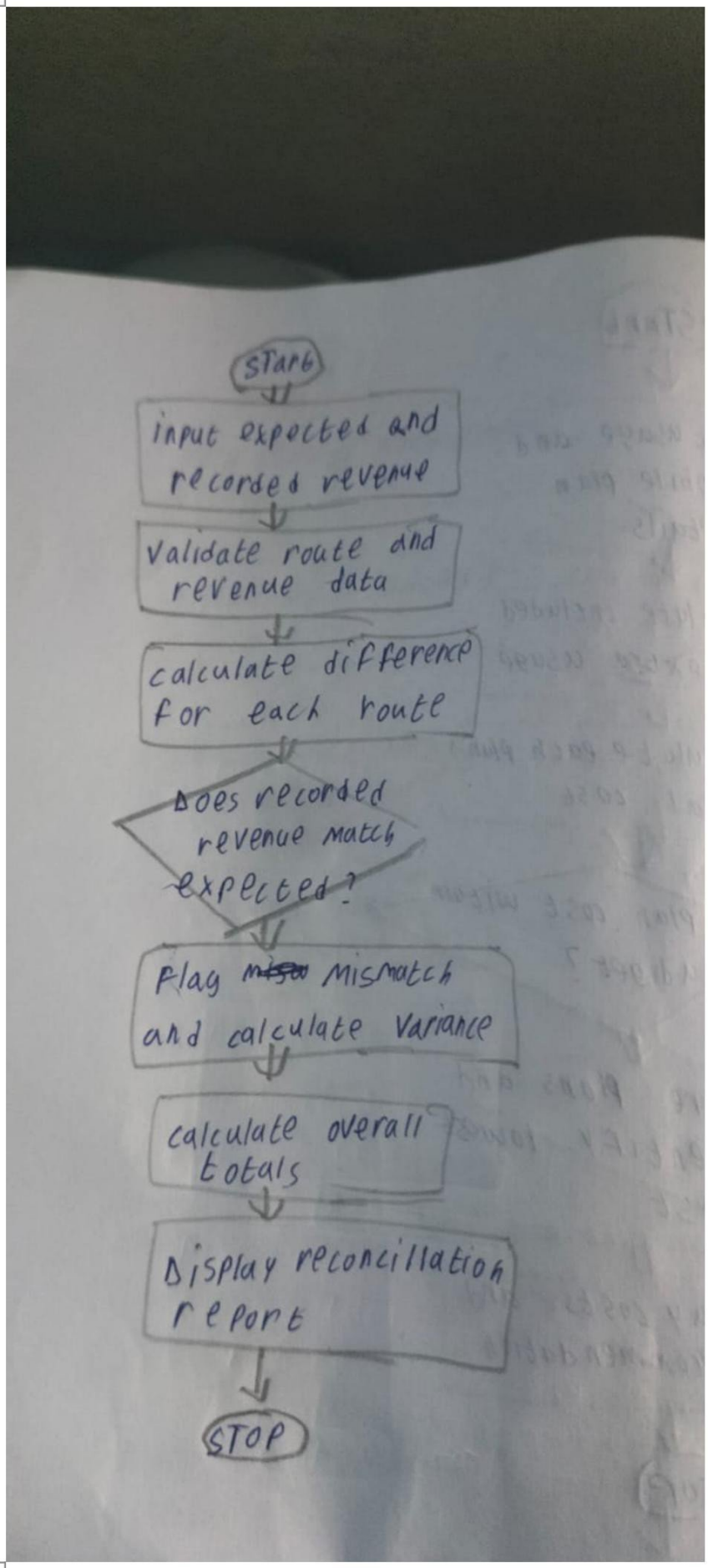
#	Test	Input	Expected result
1	Adult fare	R1, Adult	20.0
2	Student discount	R2, Student	17.5
3	Senior discount	R3, Senior	30.0
4	Child free	R1, Child	0.0
5	Unknown route	R9, Adult	None (invalid)
6	Unknown category	R1, VIP	None (invalid)
7	Expected revenue	R1: 2 Adults + 2 Students	R1 = 40 + 20 = 60.0
8	Empty ticket list	[]	All routes 0, no invalid tickets
9	Exact match	Expected 100, collected 100	MATCH
10	Within tolerance	Expected 100, collected 99.5	MATCH
11	Shortage	Expected 100, collected 80	SHORTAGE (−20)
12	Excess	Expected 100, collected 130	EXCESS (+30)
13	No collection entry	Expected 100, collected missing	SHORTAGE (−100)

Output

```
Route    Expected  Collected    Diff  Status
R1          260.0      260.0     0.0  MATCH
R2          364.0      400.0    36.0  EXCESS
R3          300.0      310.0    10.0  EXCESS

Total expected : 924.0
Total collected: 970
Net difference : 46.0
Invalid tickets: [('R9', 'Adult', 2), ('R2', 'Tourist', 1)]
```

Flowchart


START
input expected and recorded revenue
validate route and revenue data
calculate difference for each route
Does recorded revenue match expected?
Flag mismatch and calculate variance
calculate overall totals
Display reconcillation report
STOP

Conclusion

The program calculates passenger fares for each route and category, adds them up to get the expected revenue, and compares this with the revenue actually collected. It flags every route as MATCH, SHORTAGE or EXCESS and lists invalid ticket records. In the sample run, R1 matches, while R2 (Rs 36) and R3 (Rs 10) show excess collection. A real operator would investigate these, for example for unrecorded tickets or wrong fares charged. The invalid tickets (R9 and the “Tourist” category) are also listed for checking.

Problem Statement

A hospital bills each patient for three kinds of charges: **consultation**, **diagnostic** (tests, scans) and **treatment**. Some patients qualify for **concessions**, for example senior citizens and low-income patients. The bill must be checked for errors before it is issued. After the final amount is known, the patient is placed in a **payment category** that decides how they may pay.

Objectives

Store each patient's record as a tuple.

Validate the data before billing.

Calculate the subtotal from the three charges.

Find and apply the correct concession.

Calculate the net payable amount.

Classify the patient into a payment category using the policy.

Print a clear bill for every valid patient and an error message for every invalid one.

Assumptions

Each patient record is a tuple: (patient_id, name, age, low_income, consultation, diagnostic, treatment). Here low_income is True or False.

All amounts are in rupees (Rs).

Concession policy (example values):

- Senior citizen (age 60 or more): 10% off.
- Low-income patient: 20% off.
- High bill (subtotal above Rs 20,000): 5% off.

Concessions **do not stack**. Only the **highest** applicable percentage is used.

The concession can never be more than Rs 5,000.

Payment policy (based on net bill):

- Up to Rs 2,000: Category A, pay in full at the counter.
- Rs 2,001 to Rs 10,000: Category B, pay in full or in 2 installments.
- Above Rs 10,000: Category C, 50% advance deposit plus an installment plan.

Validation rules: the name must not be empty, age must be between 0 and 120, charges must not be negative, and the total bill must not be zero.

Invalid records are reported and not billed.

Algorithm

1. Define the concession rules and the payment policy.
2. Take the next patient record.
3. **Validate** it. If there are errors, print them and skip to the next patient.
4. Subtotal = consultation + diagnostic + treatment.
5. Collect every concession the patient qualifies for (senior, low-income, high bill) and take the **highest** percentage (0 if none).
6. Concession amount = subtotal × percentage / 100, limited to the maximum cap.
7. Net payable = subtotal − concession.
8. Compare the net amount with the policy limits in order and pick the first matching category.
9. Print the bill with the subtotal, concession, net amount, category and payment rule.
10. Repeat until all patients are processed.

Pseudocode

```
CONSTANTS: senior age, discount percents, high bill limit, max concession,
payment policy

FUNCTION validate_patient(patient):
  errors = empty list
  IF name is empty:              add "Name is missing"
  IF age < 0 OR age > 120:       add "Age must be between 0 and 120"
  FOR each charge:
    IF charge is not a number OR charge < 0: add error
  IF no errors AND total = 0:    add "Total bill is zero"
  RETURN errors

FUNCTION find_concession_percent(patient, subtotal):
  options = empty list
  IF age >= 60:                  add 10
  IF low_income:                 add 20
  IF subtotal > 20000:           add 5
  IF options is empty: RETURN 0
  RETURN maximum of options

FUNCTION classify_patient(net):
  FOR each (limit, category, rule) in payment policy:
    IF net <= limit: RETURN category, rule

FUNCTION process_bill(patient):
```

errors = validate_patient(patient)
IF errors not empty: RETURN invalid result
subtotal = consultation + diagnostic + treatment
percent = find_concession_percent(patient, subtotal)
concession = MIN(subtotal * percent / 100, max concession)
net = subtotal - concession
category, rule = classify_patient(net)
RETURN bill details

Python source code

```
# Hospital Billing and Patient Classification

SENIOR_AGE = 60
SENIOR_DISCOUNT = 10
LOW_INCOME_DISCOUNT = 20
HIGH_BILL_LIMIT = 20000
HIGH_BILL_DISCOUNT = 5
MAX_CONCESSION = 5000

PAYMENT_POLICY = [
    (2000, "Category A", "Pay in full at counter"),
    (10000, "Category B", "Pay in full or 2 installments"),
    (float("inf"), "Category C", "50% advance deposit + installment plan"),
]

def validate_patient(patient):
    """Return a list of billing/data errors (empty list means valid)."""
    errors = []
    if len(patient) != 7:
        return ["Wrong number of fields"]
    pid, name, age, low_income, consult, diag, treat = patient
    if not isinstance(name, str) or name.strip() == "":
        errors.append("Name is missing")
    if not isinstance(age, int) or age < 0 or age > 120:
        errors.append("Age must be between 0 and 120")
    for label, charge in [("Consultation", consult), ("Diagnostic", diag), ("Treatment", treat)]:
        if not isinstance(charge, (int, float)) or charge < 0:
            errors.append(f"{label} charge must be a non-negative number")
    if not errors and consult + diag + treat == 0:
        errors.append("Total bill is zero")
    return errors

def calculate_subtotal(patient):
    return patient[4] + patient[5] + patient[6]
```

```python
def find_concession_percent(patient, subtotal):
    """Concessions do NOT stack: the highest applicable percentage is used."""
    age, low_income = patient[2], patient[3]
    options = []
    if age >= SENIOR_AGE:
        options.append(SENIOR_DISCOUNT)
    if low_income:
        options.append(LOW_INCOME_DISCOUNT)
    if subtotal > HIGH_BILL_LIMIT:
        options.append(HIGH_BILL_DISCOUNT)
    return max(options) if options else 0

def classify_patient(net_amount):
    """Return (category, payment_rule) from the payment policy."""
    for limit, category, rule in PAYMENT_POLICY:
        if net_amount <= limit:
            return category, rule

def process_bill(patient):
    """Validate, calculate and classify. Returns a dictionary."""
    errors = validate_patient(patient)
    if errors:
        return {"id": patient[0], "valid": False, "errors": errors}
    subtotal = calculate_subtotal(patient)
    percent = find_concession_percent(patient, subtotal)
    concession = min(subtotal * percent / 100, MAX_CONCESSION)
    net = round(subtotal - concession, 2)
    category, rule = classify_patient(net)
    return {"id": patient[0], "name": patient[1], "valid": True,
            "subtotal": subtotal, "percent": percent,
            "concession": round(concession, 2), "net": net,
            "category": category, "rule": rule}
```

```python
if __name__ == "__main__":
    patients = [
        ("P01", "Ravi",    35, False,  500,  800,  400),
        ("P02", "Lakshmi", 68, False,  600, 2500, 4000),
        ("P03", "Suresh", 45, True,   500, 3000, 12000),
        ("P04", "Anil",    50, False, 1000, 8000, 30000),
        ("P05", "",        30, False,  500,  200,   100),
        ("P06", "Meena",  25, False,  500,  -300,  100),
    ]
    for p in patients:
        r = process_bill(p)
        print("Patient", r["id"], end=": ")
        if not r["valid"]:
            print("INVALID ->", "; ".join(r["errors"]))
            continue
        print(r["name"])
        print(f"  Subtotal : Rs {r['subtotal']}")
        print(f"  Concession: {r['percent']}% = Rs {r['concession']}")
        print(f"  Net bill : Rs {r['net']}")
        print(f"  Class    : {r['category']} - {r['rule']}")
```

Test Cases

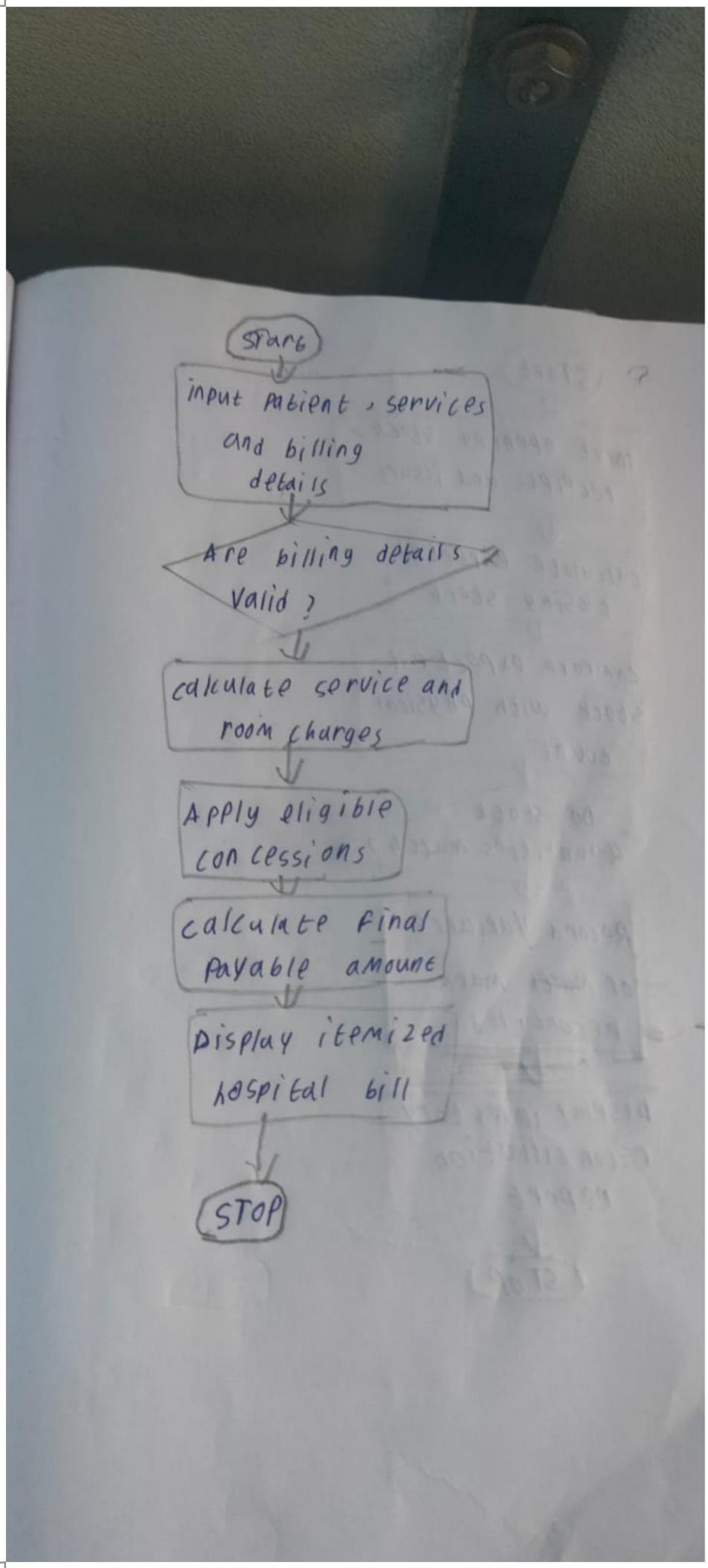
#	Test	Input	Expected result

1	No concession	Age 35, not low-income, bill Rs 1,700	0% concession, net 1700, Category A
2	Senior concession	Age 68, bill Rs 7,100	10% = 710, net 6390, Category B
3	Low-income concession	Age 45, low-income, bill Rs 15,500	20% = 3100, net 12400, Category C
4	High-bill concession	Age 50, bill Rs 39,000	5% = 1950, net 37050, Category C
5	Concessions do not stack	Senior and low-income	20% (highest only)
6	Concession cap	Senior, low-income, bill Rs 60,000	20% would be 12000, capped at 5000, net 55000
7	Boundary 2000	Net bill 2000	Category A
8	Just above 2000	Net bill 2000.01	Category B
9	Boundary 10000	Net bill 10000	Category B
10	Just above 10000	Net bill 10001	Category C
11	Missing name	Name is ""	INVALID: Name is missing
12	Negative charge	Diagnostic = −300	INVALID: charge must be non-negative
13	Bad age	Age 130	INVALID: age must be between 0 and 120
14	Zero bill	All charges 0	INVALID: Total bill is zero
15	Wrong field count	Tuple with 5 fields	INVALID: Wrong number of fields

Output

```
Patient P01: Ravi
  Subtotal : Rs 1700
  Concession: 0% = Rs 0.0
  Net bill : Rs 1700.0
  Class    : Category A - Pay in full at counter
Patient P02: Lakshmi
  Subtotal : Rs 7100
  Concession: 10% = Rs 710.0
  Net bill : Rs 6390.0
  Class    : Category B - Pay in full or 2 installments
Patient P03: Suresh
  Subtotal : Rs 15500
  Concession: 20% = Rs 3100.0
  Net bill : Rs 12400.0
  Class    : Category C - 50% advance deposit + installment plan
Patient P04: Anil
  Subtotal : Rs 39000
  Concession: 5% = Rs 1950.0
  Net bill : Rs 37050.0
  Class    : Category C - 50% advance deposit + installment plan
Patient P05: INVALID -> Name is missing
Patient P06: INVALID -> Diagnostic charge must be a non-negative number
```

Flowchart


Start
input Patient, services and billing details
Are billing details valid ?
calculate service and room charges
Apply eligible concessions
calculate final payable amoune
Display itemized hospital bill
STOP

Conclusion

The program processes consultation, diagnostic and treatment charges, validates the patient data, applies the highest eligible concession (within the cap), and classifies each patient into a payment category from the supplied policy. In the sample run, four patients were billed and placed in Categories A, B, C and C, and two records were rejected for invalid data (a missing name and a negative charge).

Problem Statement

A warehouse stores many products. Stock moves in and out every day: new goods arrive (**incoming**), and goods leave for customers or shops (**dispatched**). At the end of the period, the stock counted on the shelves (**actual closing stock**) should match what the records say it should be. If it does not, there is a **discrepancy**, which could be caused by theft, damage, miscounting or wrong entries. The warehouse also needs to know which products must be re-ordered.

Objectives

Store each product's stock data as a tuple in a list.

Validate the records before processing.

Calculate the expected closing stock for each product.

Compare it with the actual closing stock and flag OK, SHORTAGE or SURPLUS.

Calculate how many units to re-order when stock falls below the reorder level.

Print a clear reconciliation report and list the invalid records.

Assumptions

Each record is a tuple: (product, opening, incoming, dispatched, closing_actual, reorder_level, max_level).

All quantities are whole numbers (units) and cannot be negative.

Expected closing = opening + incoming − dispatched.

Difference = actual closing − expected closing. A negative difference is a SHORTAGE, a positive one is a SURPLUS, and zero is OK.

The allowed tolerance is 0 units by default, so any difference is reported. You can change this.

Reorder rule: if the actual closing stock is below the reorder level, order enough units to reach the maximum level (max_level − actual stock). Otherwise order nothing.

A record is invalid if any quantity is not a whole number, if dispatched is more than opening + incoming (you cannot dispatch stock you never had), or if max_level is below reorder_level.

Invalid records are reported and skipped.

Algorithm

1. Define the product records and the tolerance.
2. Take the next record.
3. **Validate** it. If it has errors, save the record in the invalid list and move on.

4. Calculate the expected closing stock: opening + incoming − dispatched.
5. Calculate the difference: actual − expected.
6. If |difference| <= tolerance, status is OK. If the difference is negative, it is a SHORTAGE. Otherwise it is a SURPLUS.
7. If actual closing stock is below the reorder level, replenishment quantity = max_level − actual. Otherwise it is 0.
8. Print the product's result.
9. Repeat for all records, then print the invalid records.

Pseudocode

```
tolerance = 0

FUNCTION validate_record(rec):
   errors = empty list
   FOR each quantity in rec:
      IF not a whole number OR quantity < 0: add error
   IF no errors:
      IF dispatched > opening + incoming: add "Dispatched is more than available
stock"
      IF max_level < reorder_level:      add "Max level is less than reorder level"
   RETURN errors

FUNCTION find_discrepancy(expected, actual):
   diff = actual - expected
   IF abs(diff) <= tolerance: RETURN diff, "OK"
   IF diff < 0:              RETURN diff, "SHORTAGE"
   RETURN diff, "SURPLUS"

FUNCTION replenishment_needed(actual, reorder_level, max_level):
   IF actual < reorder_level: RETURN max_level - actual
   RETURN 0

FUNCTION reconcile_product(rec):
   errors = validate_record(rec)
   IF errors not empty: RETURN invalid result
   expected = opening + incoming - dispatched
   diff, status = find_discrepancy(expected, actual)
   order = replenishment_needed(actual, reorder, max_level)
   RETURN results

MAIN:
   FOR each record: reconcile_product(record) and print
   print invalid records
```

Python source code

```python
# Warehouse Inventory Reconciliation

TOLERANCE = 0

def validate_record(rec):
    """Return a list of errors (empty list = valid)."""
    if len(rec) != 7:
        return ["Wrong number of fields"]
    name, opening, incoming, dispatched, closing, reorder, max_level = rec
    errors = []
    for label, value in [("Opening", opening), ("Incoming", incoming),
                         ("Dispatched", dispatched), ("Closing", closing),
                         ("Reorder level", reorder), ("Max level", max_level)]:
        if not isinstance(value, int) or value < 0:
            errors.append(f"{label} must be a whole number >= 0")
    if not errors:
        if dispatched > opening + incoming:
            errors.append("Dispatched is more than available stock")
        if max_level < reorder:
            errors.append("Max level is less than reorder level")
    return errors

def expected_closing(opening, incoming, dispatched):
    return opening + incoming - dispatched

def find_discrepancy(expected, actual, tolerance=TOLERANCE):
    """Return (difference, status). Difference = actual - expected."""
    diff = actual - expected
    if abs(diff) <= tolerance:
        return diff, "OK"
    if diff < 0:
        return diff, "SHORTAGE"
    return diff, "SURPLUS"
```

```python
def replenishment_needed(actual, reorder_level, max_level):
    """If stock is below the reorder level, order enough to reach max level."""
    if actual < reorder_level:
        return max_level - actual
    return 0

def reconcile_product(rec):
    errors = validate_record(rec)
    if errors:
        return {"product": rec[0], "valid": False, "errors": errors}
    name, opening, incoming, dispatched, closing, reorder, max_level = rec
    exp = expected_closing(opening, incoming, dispatched)
    diff, status = find_discrepancy(exp, closing)
    order = replenishment_needed(closing, reorder, max_level)
    return {"product": name, "valid": True, "expected": exp, "actual": closing,
            "diff": diff, "status": status, "reorder": order}

if __name__ == "__main__":
    records = [
        ("Rice",   500, 200, 450, 250, 150, 600),
        ("Oil",    300, 100, 350,  40, 100, 400),
        ("Sugar",  200,  50, 100, 160, 100, 300),
        ("Salt",   100,  20, 150,  10,  30, 200),
        ("Tea",     80,  40,  60,  -5,  20, 100),
    ]
    print(f"{'Product':<8}{'Expected':>9}{'Actual':>8}{'Diff':>6}  {'Status':<9}{'Order Qty':>9}")
    invalid = []
    for rec in records:
        r = reconcile_product(rec)
        if not r["valid"]:
            invalid.append(r)
            continue
        print(f"{r['product']:<8}{r['expected']:>9}{r['actual']:>8}{r['diff']:>6}  {r['status']:<9}{r['reorder']:>9}")
    print()
    for r in invalid:
        print("INVALID record:", r["product"], "->", "; ".join(r["errors"]))
```

Test Cases

#	Test	Input	Expected result
1	Expected closing	Opening 500, incoming 200, dispatched 450	250
2	Perfect match	Expected 250, actual 250	Difference 0, OK
3	Shortage	Expected 250, actual 240	Difference −10, SHORTAGE
4	Surplus	Expected 250, actual 260	Difference +10, SURPLUS
5	Below reorder level	Actual 40, reorder 100, max 400	Order 360
6	Exactly at reorder level	Actual 100, reorder 100, max 400	Order 0
7	Just below reorder level	Actual 99, reorder 100, max 400	Order 301

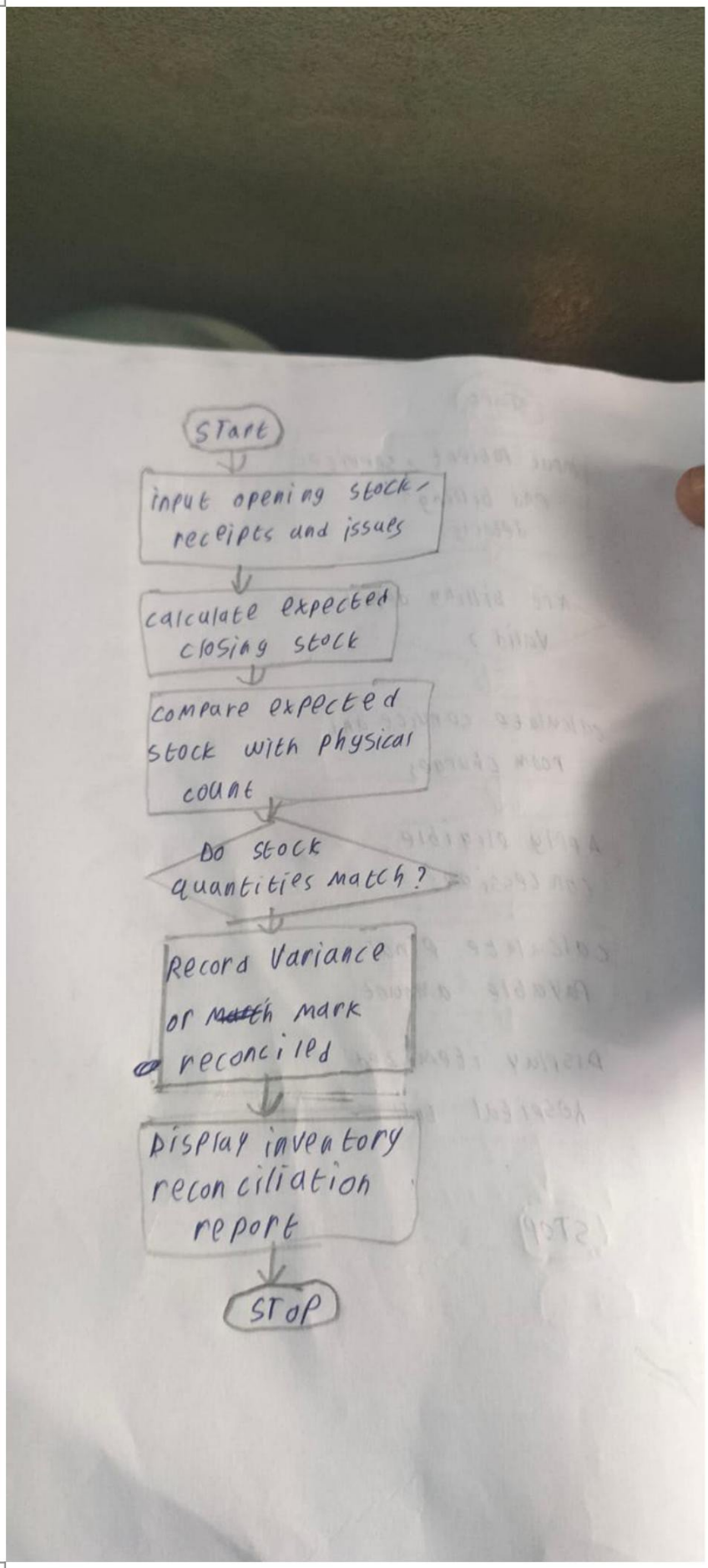
8	Zero stock	Actual 0, reorder 10, max 50	Order 50
9	Over-dispatch	Opening 10, incoming 5, dispatched 20	INVALID: dispatched is more than available stock
10	Max below reorder	Reorder 50, max 20	INVALID: max level less than reorder level
11	Bad quantity	Closing = 0.5 or −5	INVALID: must be a whole number >= 0
12	Wrong field count	Tuple with 3 fields	INVALID: wrong number of fields
13	All zeros	Every quantity 0	Valid, expected 0, OK, order 0

Output

```
Product  Expected  Actual  Diff  Status    Order Qty
Rice          250     250     0  OK                0
Oil            50      40   -10  SHORTAGE        360
Sugar         150     160    10  SURPLUS           0

INVALID record: Salt -> Dispatched is more than available stock
INVALID record: Tea -> Closing must be a whole number >= 0
```

Flowchart


START
input opening stock, receipts and issues
calculate expected closing stock
compare expected stock with physical count
Do stock quantities match?
Record Variance or mark reconciled
Display inventory reconciliation report
STOP

Conclusion

The program compares opening stock, incoming, dispatched and closing stock for several products. It flags discrepancies as SHORTAGE or SURPLUS, rejects impossible records, and calculates how many units to re-order for products running low. In the sample run, Rice matched exactly. Oil had a shortage of 10 units and needs a re-order of 360 units. Sugar had a surplus of 10 units. Salt and Tea were rejected because of bad data (dispatching more than was available, and a negative closing count).